The Journal of Systems and Software 231 (2026) 112626

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Multivariate anomaly detection and root cause analysis of energy issues in microservice-based systems☆

Berta Rodriguez Sanchez [a], Luca Giamattei [b], Antonio Guerriero [c], Roberto Pietrantuono [b], Ivano Malavolta [a,*]

[a] Vrije Universiteit Amsterdam, The Netherlands
[b] University of Naples Federico II, Naples, Italy
[c] University of Salerno, Fisciano (SA), Italy

ARTICLE INFO

Keywords:
Software architecture
Anomaly detection
Root cause analysis
Microservices
Energy

ABSTRACT

Context: Microservice-based systems have become the architecture style of choice for modern applications, offering scalability, flexibility, and resilience. However, their distributed nature leads to increased resource consumption and energy inefficiencies, posing challenges for maintaining sustainable operations. Accurate anomaly detection (AD) and root cause analysis (RCA) tools are critical for diagnosing energy consumption issues in these systems, yet existing solutions often lack focus on energy metrics.

Goal: This study aims to evaluate the effectiveness of AD and RCA algorithms in identifying and diagnosing performance-related energy consumption anomalies in microservice-based systems.

Method: Two representative systems, Sock Shop and Train Ticket, are deployed under controlled environments. Then, anomalies are deliberately introduced by stressing at the same time CPU, memory, and disk resources. The data collection is conducted using Prometheus for performance metrics and Scaphandre for energy metrics. Once normal and anomalous datasets are constructed for each system, the study evaluates five AD algorithms (Birch, iForest, KNN, LOF, and SVM) and four RCA algorithms (MicroRCA, CausalRCA, CIRCA, and RCD) based on their precision, recall, and scalability across varied scenarios and workloads.

Results: The experiment reveals that overall, iForest is the most effective AD algorithms in detecting energy anomalies (0.59 F-Score in Sock Shop and 0.634 F-Score in Train Ticket). In particular, iForest performs better in precision when the user load is high (1000 concurrent users). For RCA, CIRCA performs well in identifying root causes in smaller systems, while RCD is more scalable for larger and more complex systems.

Conclusions: The findings of this study provide insights for both researchers and practitioners. In the context of our experiment, AD algorithms tend to perform relatively well, whereas RCA algorithms tend to be imprecise in localizing energy issues.

1. Introduction

The rapid evolution of software systems has established microservice architecture style as a preferred choice of modern application design (Shaw and Garlan, 1996). Companies like Amazon, Netflix, and Spotify have adopted microservice-based systems into their core business (Soldani et al., 2018). Microservices are small, loosely coupled components that can be independently deployed, enabling systems to achieve high scalability, flexibility, and resilience. However, this shift to distributed systems introduces challenges, particularly the increased consumption of network and computing resources compared to other architectural styles (Soldani et al., 2018). This issue also concerns energy efficiency, an important aspect of today's IT industry (Araújo et al., 2024; Hameed et al., 2016).

As organizations increasingly rely on cloud computing and microservice-based systems, optimizing energy efficiency has become crucial for both environmental and economic reasons (Pinto and Castor, 2017; Vos et al., 2022). The IT sector is a significant contributor to global energy consumption (Freitag et al., 2021), with data centers alone accounting for approximately 1–1.5% of global electricity use.[1]

☆ Editor: Neil Ernst.

* Corresponding author.
E-mail addresses: b.rodriguezsanchez@student.vu.nl (B.R. Sanchez), luca.giamattei@unina.it (L. Giamattei), antguerriero@unisa.it (A. Guerriero), roberto.pietrantuono@unina.it (R. Pietrantuono), i.malavolta@vu.nl (I. Malavolta).

[1] https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks.

https://doi.org/10.1016/j.jss.2025.112626
Received 9 April 2025; Received in revised form 25 July 2025; Accepted 5 September 2025
Available online 18 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Understanding and managing the growing energy demands of IT systems, especially in costly technologies like artificial intelligence, is key for reducing their operational costs and energy footprint (de Vries, 2023).

Monitoring and diagnosing energy inefficiencies in microservice-based systems is a complex task not only due to the non-trivial interconnectedness of services but also because each service can have its own technological stack, be deployed differently, and exhibit varying life spans and resource usage patterns. Anomalous CPU or memory usage observed in a particular service may reflect changes in request patterns or shifts in resource demands across interconnected services. Such variations can lead to elevated loads or bottlenecks, subsequently constraining downstream activities. These complex interactions significantly influence the overall energy consumption of the system, making the effective detection and diagnosis of inefficiencies not trivial at all. Therefore, it is imperative to investigate the efficacy of anomaly detection and root cause analysis techniques specifically tailored to address issues related to energy consumption in distributed computing environments. Anomaly detection (AD) algorithms are used in production by DevOps teams to monitor system behavior and identify deviations from normal behavior, such as resource spikes or latency issues (Shahin et al., 2023; Kamat and Sugandhi, 2020). Root cause analysis (RCA) algorithms complement AD by tracing these anomalies back to their underlying causes, enabling teams to address these issues (Latino et al., 2019; Steenwinckel et al., 2021). Both AD and RCA can be effective in improving system reliability and minimizing downtime in real-world applications (Abdelrahman and Keikhosrokiani, 2020; Shahin et al., 2023; Latino et al., 2019).

The **goal of this study** is to *investigate the effectiveness of AD and RCA algorithms in detecting and diagnosing energy consumption anomalies caused by performance issues in the context of microservice-based systems*. This is done by deliberately introducing performance-related energy anomalies, such as increased CPU and memory usage in a service, to assess the capabilities of these algorithms. Two microservice-based systems, Sock Shop,[2] which has 13 services, and Train Ticket,[3] which has 41 services, serve as testbeds for the experiments. The systems are monitored using Prometheus,[4] which collects service-level metrics such as CPU and memory utilization, and Scaphandre,[5] which collects service-level power consumption data about CPU and DRAM, later converted to energy consumption. The AD and RCA algorithms used in this study are selected from the existing literature, focusing on those that can handle multivariate data, require little processing, and have publicly available code to ensure integration with collected performance and energy metrics.

The **main results** of this study highlight the importance of selecting AD and RCA algorithms that align well with energy-related metrics and the scale of the system under consideration, as their accuracy varies significantly with dataset and workload characteristics. Specifically, our results for AD algorithms indicate that the evaluated algorithms are promising candidates for energy-related anomaly detection. Among the considered AD algorithms, iForest tends to be the most consistent and reliable, achieving the highest F-score across most configurations; BIRCH shows strong precision, particularly in Sock Shop, but its low recall limits its overall effectiveness, whereas KNN, LOF, and SVM exhibit high recall but suffer from low precision. RCA algorithms perform less effectively; RCD is the most robust RCA algorithm, consistently delivering relatively good results across both systems by leveraging scalable causal discovery, still with relatively low accuracy levels; CIRCA achieves relatively high precision in Sock Shop but struggles with scalability in larger systems like Train Ticket. We obtained mixed results for MicroRCA, while CausalRCA frequently misidentifies energy-related root causes across the board.

The **main contributions** of this study are summarized below:

⋆ **Datasets:** Providing datasets of both the normal and anomalous behavior in the Sock Shop and Train Ticket systems. The datasets include performance metrics such as CPU and memory usage, as well as CPU/DRAM energy consumption data, collected over 2.5 days of continuous experiments. The data consists of thousands of observations per system (21 000+ observations per system including the normal and anomalous datasets), which cover various usage scenarios, services under stress, and user loads. Therefore, the datasets can be useful to researchers as benchmarks or input data for other algorithms. We release an open repository[6] containing the datasets and instructions to replicate our experiments.

⋆ **Algorithm Evaluation (AD):** Evaluating the effectiveness of AD algorithms in identifying energy consumption issues in microservice-based systems.

⋆ **Algorithm Evaluation (RCA):** Evaluating the effectiveness of RCA algorithms in identifying the root cause of energy consumption issues in microservice-based systems.

⋆ **In-depth Discussion:** Providing a detailed discussion of the obtained results. It identifies patterns in how algorithms perform across different scenarios and systems, offering practical takeaways. For researchers, the findings show some challenges that need to be addressed, such as scalability and better integration of energy metrics. For DevOps engineers, the results provide useful strategies for detecting and diagnosing anomalies in real-world systems.

In Floroiu et al. (2024) we carried out a preliminary experiment on the accuracy of AD and RCA algorithms in identifying energy issues in microservice-based systems. In this article, we extend the contributions of that study as follows: (i) we make the algorithm selection protocol (and its reporting) more systematic, while ensuring that the selected algorithms are only multivariate ones, *i.e.,* they can analyze both performance and energy consumption metrics at the same time, (ii) as a result of the previous point, we consider 3 additional AD and RCA algorithms (*i.e.,* LOF, CausalRCA, and CIRCA), while discarding two out-of-scope algorithms, (iii) we replaced the closed-source system used in Floroiu et al. (2024) with an open-source one (*i.e.,* Train Ticket), (iv) we strengthen the statistical power of the performed experiment by repeating each experimental trial 30 times (as suggested in state-of-the-art guidelines on energy-related experiments (Guldner et al., 2024)) instead of only 10 times, and (v) we extend the description and discussion of the obtained results by considering the architectural aspects of the measured systems, instead of focussing exclusively on purely quantitative results related to the accuracy of the algorithms (*e.g.,* accuracy, precision, F-score).

2. Background

2.1. Microservice architecture style

The microservice architecture style is an approach to software development where large, monolithic applications are decomposed into smaller, independent services (Nadareishvili et al., 2016). Since each service operates autonomously and is only in charge of one function, a failure in one service is unlikely to propagate throughout the system, making the system more reliable and stable (Kalia et al., 2021). Moreover, these loosely coupled services communicate with each other through APIs, often by using lightweight protocols such as HTTP (Newman, 2021). This design allows each individual service to be developed,

[2] https://github.com/microservices-demo/microservices-demo.
[3] https://github.com/FudanSELab/train-ticket.
[4] https://prometheus.io/.
[5] https://hubblo-org.github.io/scaphandre-documentation/index.html.
[6] https://github.com/uDEVOPS2020/Multivariate-AD-and-RCA-of-Energy-Issues-in-Microservice-based-Systems.

deployed, and scaled independently, which improves flexibility and maintainability (Dragoni et al., 2017). If a specific service experiences a spike in transactions, instead of the entire application being affected like in monolithic systems, only that service would need additional resources (Di Francesco et al., 2019). When it comes to deploying microservice-based systems, there are multiple containerization technologies available, with one of the most used being Docker[7] (Jaramillo et al., 2016; Rufino et al., 2017). Docker enables each microservice to be run as a container, which is a lightweight, portable environment containing all the service's necessary dependencies and configurations (Wan et al., 2018). This provides consistent environments across the development, testing, and production phases of the system.

2.2. Energy-efficient microservices

The energy efficiency in microservice-based systems is one of the aspects that may impact operational costs. On the one hand, the independent scalability of such systems could lead to better energy utilization than monolithic systems. This can lead to better energy utilization by focusing resources where they are most needed. On the other hand, there are other energy inefficiencies introduced by the distributed nature of microservice-based systems. The need for frequent communication between services can increase energy consumption. Additionally, if each service runs on separate virtual machines or containers, it can lead to inefficiencies when services are idle. Santos et al. (2018) analyzed the impact of Docker containers on energy consumption, finding that Docker introduces energy overhead, mainly due to increased performance of I/O system calls. Dinga et al. (2023) evaluated the energy and performance overhead of monitoring tools on Docker-based systems, proving how even monitoring tasks add to the overall energy consumption.

2.3. Anomaly detection

Anomaly detection (AD) is the process of finding patterns in data that significantly differ from the defined normal behavior (Chandola et al., 2009). A common anomaly detection method is to establish a normal behavior and label any instance in the data that significantly deviates from this normal behavior as anomalous. Machine learning techniques are often used for anomaly detection, and can be of three categories: supervised AD, which requires that the dataset has labeled normal and anomalous instances and involves training a classifier; semi-supervised AD, which assumes that the dataset has labeled normal instances; and unsupervised AD, which do not require labeled data (Chandola et al., 2009). Anomaly detection has a wide variety of applications such as health care, cybersecurity, and fault detection in safety-critical systems.

In the context of microservice-based systems, anomaly detection usually involves monitoring system-level or service-level metrics to detect unexpected behaviors. Common metrics include CPU usage, memory consumption, response times, and, in our study, energy consumption. If an anomaly occurs in a service, such as an increase in resources, a latency spike, or a sudden failure, it will be shown in the metrics monitored.

For example, consider a microservice-based system like Sock Shop where the front-end service suddenly experiences a high number of requests, causing a bottleneck. An AD algorithm such as iForest (Liu et al., 2008) could analyze metrics like energy consumption over time to detect the anomaly. Fig. 1 illustrates an example of this process, showing the sudden spikes in energy consumption for the front-end service compared to its normal behavior. The iForest algorithm labels these spikes as anomalies due to their deviation from the normal behavior of energy consumption.

[7] https://docs.docker.com/get-started/docker-overview/.

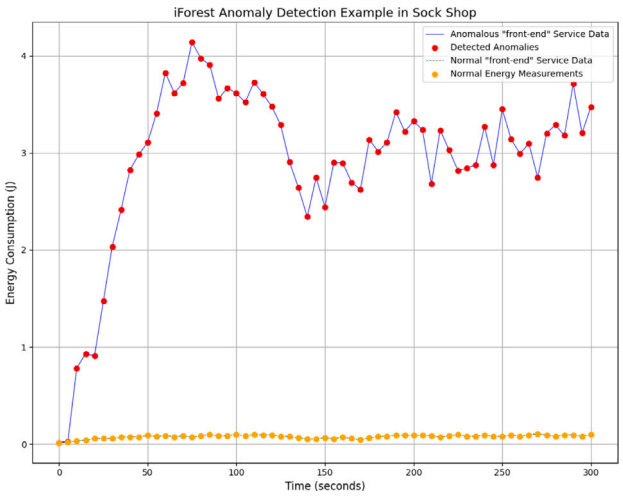


Fig. 1. Anomaly detection example in sock shop.

2.4. Root cause analysis

Root cause analysis (RCA) is a problem-solving method for identifying the underlying reasons of a detected anomaly within a system. Most RCA techniques share a general workflow, which consists of first building a model from observed data and sometimes domain or system knowledge of the diagnosed system, which will then be used to infer the root causes (Solé et al., 2017). Some RCA techniques include direct analysis, which directly processes the data to determine other services that could have caused the anomaly, and topology or causal graph-based analysis, which model the dependencies in a system and determine the most likely cause by walking through the graph (Soldani and Brogi, 2022). Once the root cause of a problem is determined, developers should implement preventive measures so that similar issues do not happen again (Peerally et al., 2017). In microservice-based systems, RCA is challenging due to the interconnectedness of services and dynamic network interactions. When an anomaly occurs in a specific service, it may propagate to other services it interacts with, leading to complex propagation patterns. Therefore, tracing the anomaly back to the root cause may prove to be difficult for RCA algorithms.

For example, in the same case described in Section 2.3, an RCA algorithm such as CIRCA (Li et al., 2022) would begin by generating a causal graph that reflects the cause–effect relationships between variables within the system. Fig. 2 provides an example of a causal graph generated by CIRCA using energy consumption data from the Sock Shop system. In this study, however, the RCA algorithms analyze not only energy consumption but also CPU and memory usage across services, making the causal graph more complex than the one shown in Fig. 2. Using this graph, the CIRCA algorithm can infer the potential root causes of observed anomalies.

3. Related work

The study by Floroiu et al. (2024) addresses the growing concern of energy consumption in cloud computing and microservice-based architecture styles. The study experimentally evaluates the effectiveness of algorithms for anomaly detection (AD) and root cause analysis (RCA) in managing energy efficiency within containerized microservices. By analyzing five AD and three RCA methods, the paper contributes to understanding how to mitigate environmental impacts while ensuring the scalability and flexibility of microservices systems. As already anticipated, this study is based on the work by Floroiu et al. with the primary differences described in Section 1. Moreover, the results in Floroiu et al. (2024) confirm that AD and RCA algorithms have relatively

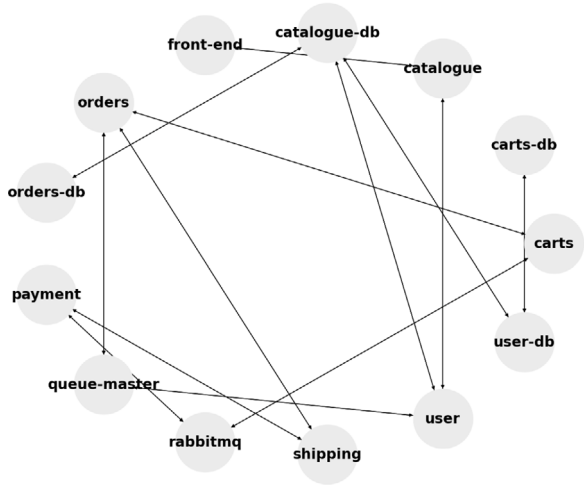


Fig. 2. Causal graph example in sock shop.

consistent results across different Prometheus scraping intervals, with more reliable results with a 5-s interval; in this study we fixed the Prometheus scraping interval to 5 s based on this result, allowing us to have the scraping interval as a fixed factor in this experiment, which in turn allowed us to keep the experiment execution time to a reasonable duration. Similarly, based on the results in Floroiu et al. (2024), we decided to fix the stress level to 90% and to have only two levels for the user load independent variable.

Solé et al. (2017) address the challenges of automating root-cause analysis in the era of large, distributed systems such as those in cloud computing and IoT environments. The authors explore how understanding the root causes of observed symptoms has become increasingly critical given the exponential growth of data generation. The survey provides a comprehensive overview of existing models and techniques, focusing on their applicability to massive datasets and their potential to support decision-making in complex systems. In the current study, one of the systems under test has a complex architecture and its results serve as a good dataset to evaluate the performance of RCA algorithms.

Soldani and Brogi (2022) systematically examine techniques for identifying anomalies and diagnosing root causes in microservice and cloud-native applications. The paper highlights the challenges of detecting failures and tracing their origins as soon as possible. It provides a structured overview and qualitative analysis of existing methods, focusing on monitoring and log analysis to uncover root causes. The current study differs from this paper in that the AD and RCA algorithms chosen from existing literature are evaluated based on energy consumption metrics.

4. Goal and research questions

The scope of the experiment consists of applications with a microservice-based architecture style with a focus on energy consumption issues directly caused by an abnormal usage of a hardware resource (e.g., CPU, memory, disk). In this context, a performance issue is defined as a significant deviation in system metrics such as high CPU and memory usage. Similarly, an *energy issue* refers to an unexpected increase in total energy consumption of the CPU package (including the CPU cores and integrated graphics if present) and DRAM that differs from the normal operation of a service. These performance and energy consumption metrics are used to execute anomaly detection and root cause analysis in microservice-based systems. The aim of our study is to evaluate the performance of AD and RCA algorithms in identifying and localizing energy issues. We use the GQM approach (Caldiera and Rombach, 1994) to define the goal, research questions, and used metrics of our study. The formal description of the goal of the experiment is as follows: "Analyze *AD and RCA algorithms* for the purpose of *evaluation* with respect to *energy consumption issues* from the point of view of *practitioners and researchers* in the context of *microservice-based applications*". A visual representation of the GQM is shown in Fig. 3, which includes the following two research questions (RQ) of the study.

RQ1: What is the effectiveness of **AD algorithms** in detecting energy consumption issues in microservice-based applications? This question is focused on assessing how well AD algorithms are able to detect when the energy consumption experiences an abnormal increase in a service of microservice-based systems. For practitioners, such as DevOps engineers, addressing this question ensures that the AD algorithms they use can reliably detect anomalies before they become larger issues. For researchers, answering this question can provide insights into the strengths and limitations of existing AD algorithms, especially in detecting anomalies in energy consumption, an important aspect that is not common in microservice research (See Section 2.2).

RQ2: To what extent can **RCA algorithms** support the identification of services being the root cause of energy issues in microservice-based systems? This RQ investigates how well RCA algorithms trace the energy consumption issues back to the original cause, which were the performance metrics of microservice-based systems. For practitioners, effective RCA reduces unnecessary resource consumption, troubleshooting time, and ensures system reliability. For researchers, this question helps evaluating current RCA algorithms' ability to integrate energy metrics into their analysis and identifies opportunities for improving RCA algorithms.

To assess the performance of AD and RCA algorithms, three different metrics are chosen in the following categories: correctness, used for both AD and RCA algorithms, completeness, and accuracy.

Correctness. The correctness of the AD and RCA results is evaluated using the precision metric. This metric is used to assess the percentage of anomalies or root causes correctly identified. For the AD algorithms, the correct anomalies are the ones related to energy consumption, whereas for the RCA algorithms, the correct root causes are in the performance metrics. Precision is used for AD algorithms, while for RCA algorithms, the precision of the top k causes, represented by P@k, is chosen as well as the average of the precision at top k, denoted as AP@k. If the correctness of these algorithms is low, it is likely that they were not effective in identifying the energy anomalies.

Completeness. The completeness of AD algorithms is measured by the recall metric. Recall reveals the ability of AD algorithms to identify relevant results. When recall is low, it means that there were more anomalies that were not detected at all (false negatives) than correctly identified anomalies (true positives). This could cause a significant problem for practitioners, as undetected anomalies (false negatives) can escalate into more threatening system issues, leading to downtime, worse performance, or increased operational costs.

Accuracy. The accuracy of AD results is assessed by the F-score metric. This metric is a combination of the precision and recall of the AD algorithms, specifically the harmonic mean of the two. It is particularly useful when there is an imbalance between the positive samples (anomalous cases) and negative samples (normal system behavior). In this study, significantly more normal instances are expected than anomalous ones. An instance refers to a single data point or measurement of metrics collected during the experiment, such as CPU usage, memory usage, or energy consumption. A low F-score indicates that the overall performance of the AD algorithm is unsatisfactory, failing to effectively balance between identifying anomalies accurately and capturing all anomalies present. For practitioners, this means that the algorithm may either cause too many false alarms, or fail to detect critical issues. For researchers, a low recall F-score means that there is a need for improved algorithms that are reliable in imbalanced datasets, which is common in real-world systems.

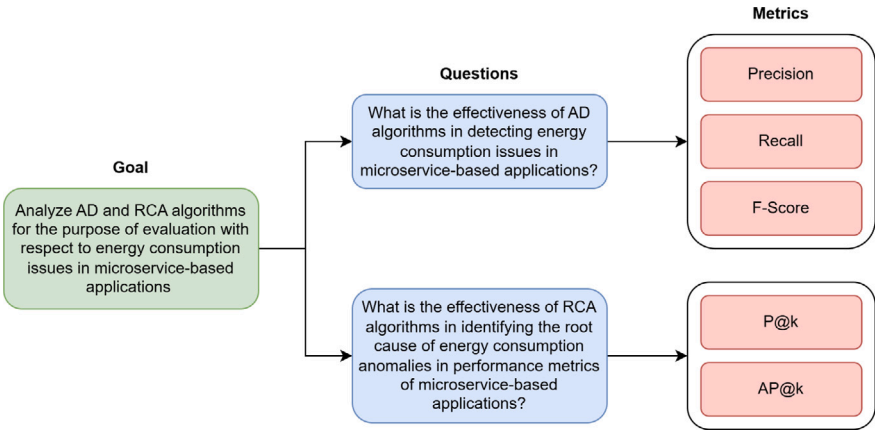


Fig. 3. Visual representation of GQM.

Table 1
Comparison of the Sock Shop and Train Ticket systems.

Metrics	Sock Shop	Train Ticket
Year of creation	2016	2019
Time of last release	2021	2022
Number of microservices	13	59
Programming languages	4	6
Types of databases	1	2
Communication protocol	HTTP	HTTP
Commits	1705	323
Contributors	56	26
Forks	2800+	249
Stars	3700+	726
Lines of code	15 049	343 167

5. Experiment plan

5.1. Systems under test

In this study, two microservice-based systems of different complexity levels will be tested and observed. Table 1 depicts different metrics that can be used to compare these two systems. The number of microservices, programming languages, types of databases, and lines of code show that the Train Ticket system is larger and more complex compared to Sock Shop. On the other hand, the number of forks and stars show that Sock Shop is used more often (typically in research contexts).

Sock Shop is a microservice demo application that simulates an e-commerce application for an online sock store. As illustrated in Fig. 4 the straightforward architecture makes it a widely used microservice-based system to test new algorithms and technologies (Ikram et al., 2022; Wu et al., 2020; Lin et al., 2018). Edge Router serves as a proxy to route traffic to the front-end service, colored green. The symbols in the left corner of some services indicate that there is an additional service for the corresponding database. The front-end service is the entry point for user interactions, the user service handles user authentication, the catalogue service provides detailed information about the socks available, the cart service allows users to add, update, or remove items, and the orders service processes the transaction through the payment and shipping services. These services are loosely coupled and communicate via the HTTP protocol. The programming languages and frameworks used in Sock Shop are Spring Boot, Go, and Node.js. Sock Shop's comprehensive design and realistic services make it an ideal subject for this experiment.

Train Ticket is one of the most used benchmarks for microservice-based systems (Zhou et al., 2018) that simulates a train ticket booking webpage. In Fig. 5, it shows the architecture of the Train Ticket system, where the ts-ui-dashboard-service provides all the user interface to interact with the system, using Nginx[8] to route traffic to the services colored green. Those services with a symbol in the left corner have an additional database service with the same name. Some of the most relevant services in this system include: ts-auth-service, which manages user data and authentication operations; ts-travel-service, which manages the train's trips available; ts-order-service, which manages the orders made; and ts-preserve-service, which preserves the train's ticket information. Each service communicates with each other via REST API. The programming languages and frameworks used in Train Ticket are Spring Boot, Django, Go, MongoDB, and MySQL. Compared to Sock Shop, Train Ticket is significantly more complex, so including it as a subject under test in the experiment will provide a new perspective to the results.

5.2. Algorithms selection

In this section, existing AD and RCA algorithms in the literature are described, as well as the process of selecting the ones that will be evaluated.

The AD and RCA algorithms are evaluated against a series of requirements, which are as follows:

1. Can handle multivariate data: The data collected contains not only energy metrics but also performance metrics. Algorithms must be capable of processing multiple variables as input.
2. Code available: The code must be publicly accessible in GitHub repositories so it can be downloaded without needing to contact the authors, request special permissions, or navigate broken links. This ensures reproducibility and reduces any delays.
3. Minimal processing: The algorithm should accept the collected data as input with only basic format adjusting, such as filtering the relevant columns for the analysis.
4. Easy to use: An algorithm is considered "easy to use" if an experienced developer can understand and execute it within a few hours of reviewing the provided documentation.

5.2.1. AD algorithms

The AD algorithms considered are chosen by first investigating the algorithms used by Floroiu et al. (2024) that performed well. Then, the AD algorithms in the survey by Soldani and Brogi (2022) are assessed to understand whether they are relevant to the study. The algorithms considered are only multivariate ones to ensure that they can analyze not only performance metrics but also energy consumption. TraceArk (Zeng et al., 2023) was not chosen in the end because it does not have available code. Moreover, the algorithm by Torabi et al. (2023)

[8] https://nginx.org/en/.

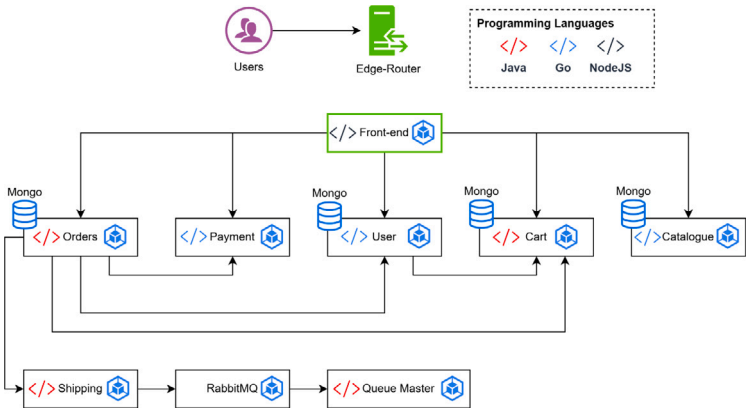


Fig. 4. Sock shop architecture. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

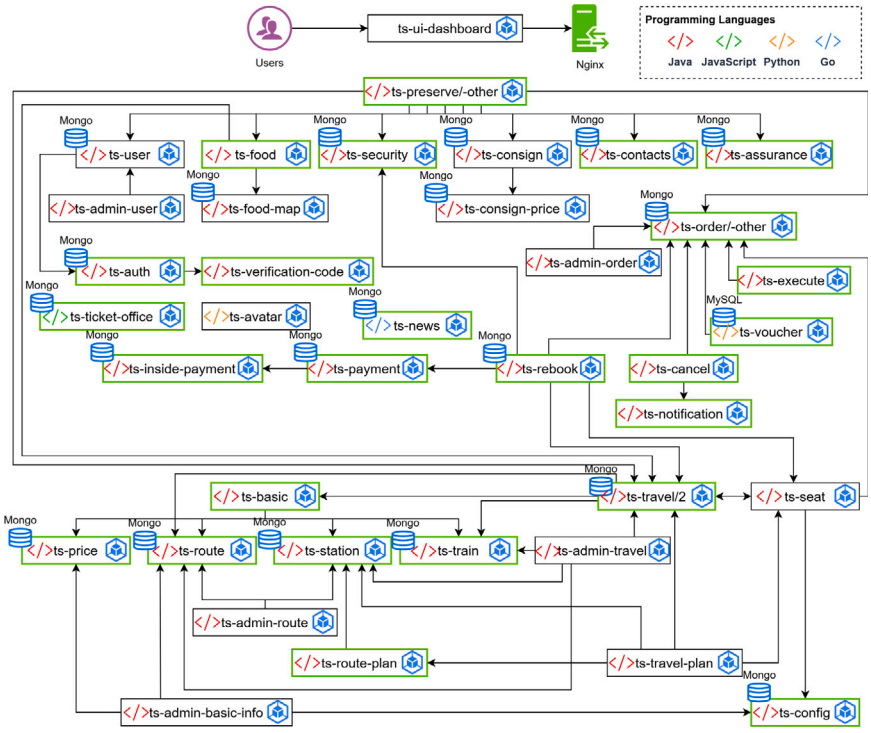


Fig. 5. Train ticket architecture. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

needs aspects in the input that are not part of the collected data of the experiment, such as the source and destination IP address, so it could not be used.

Therefore, as shown in Table 2, the algorithms selected are: Birch (Zhang et al., 1996) from the implementation in Wu et al. (2020), an unsupervised clustering algorithm which uses hierarchies; isolation forest (iForest), an unsupervised anomaly detection algorithm that isolates data points in a random forest of decision trees; a semi-supervised binary classification algorithm that uses kernels to model non-linear relationships; local outlier factor (LOF), an unsupervised anomaly detection algorithm that measures the local density deviation of a data point relative to its neighbors; and k-nearest neighbors (KNN), an unsupervised classification algorithm that classifies a point based on the majority class of its k nearest neighbors.

5.2.2. RCA algorithms

In order to choose which RCA algorithms to consider for this study, first the algorithms in the paper by Floroiu et al. (2024) that performed well were investigated. Then, the ones that can handle multivariate data and are compatible with resource-level metrics in the survey by Soldani and Brogi (2022) are considered. Additionally, to ensure that more recent RCA algorithms are included as candidates, the literature that cited the survey by Soldani et al. is also assessed. However, some algorithms such as CORAL (Wang et al., 2023), MicroScope (Lin et al., 2018), Seer (Gan et al., 2019b), AutoMAP (Ma et al., 2020), and CauseInfer (Chen et al., 2014) do not have any code available. Moreover, Nezha (Yu et al., 2023) needs additional information as input that the collected data does not include, such as logs and traces, so it is not selected.

Table 3 shows the RCA algorithms that were selected in the end. MicroRCA (Wu et al., 2020) uses a topology graph that models anomaly propagation across services to infer the root causes. CausalRCA (Xin et al., 2023) uses gradient-based causal learning to generate weighted causal graphs and from it identifies the root causes. Root Causal Discovery (RCD) (Ikram et al., 2022) is a causal graph-based algorithm that hierarchically explores the graph generated to find the root causes. CIRCA (Li et al., 2022) is a causal inference-based algorithm that uses

Table 2
Anomaly detection algorithms used in this study.

Algorithm	Year	Type	Characteristics
Birch (Zhang et al., 1996)	1996	Unsupervised	Hierarchical clustering, Scalable
iForest (Liu et al., 2008)	2008	Unsupervised	Isolation-based, Scalable
OC-SVM (Shin et al., 2005)	2005	Semi-supervised	Binary classification, Kernel-based, Sensitive
LOF (Breunig et al., 2000)	2000	Unsupervised	Outlier detection, Density-based, Sensitive
KNN (Cover and Hart, 1967)	1967	Unsupervised	Classification, Instance-based, Sensitive

Table 3
Root cause analysis algorithms used in this study.

Algorithm	Year	Type	Characteristics
MicroRCA (Wu et al., 2020)	2020	Topology graph-based analysis	Random walk, Models anomaly propagation, Lightweight
CausalRCA (Xin et al., 2023)	2023	Causal graph-based analysis	Gradient-based, Fine-grained
RCD (Ikram et al., 2022)	2022	Causal graph-based analysis	Hierarchical learning, Lightweight, Scalable
CIRCA (Li et al., 2022)	2022	Causal graph-based analysis	Cause inference, Regression-based, Descendant adjustment

Table 4
Co-independent variables.

Usage scenario	Service stressed	User load
Scenario A	Service 1 (front-end/ts-orders-service)	100 users
Scenario B	Service 2 (orders/ts-travel-service)	1000 users
–	Normal operations	–

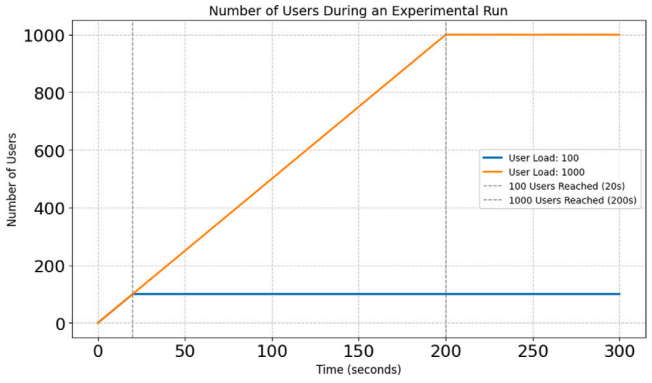


Fig. 6. Users being gradually added.

regression-based hypothesis testing and descendant adjustment to find the root causes.

5.3. Experiment variables

For RQ1, the main variables are the AD algorithms utilized, namely Birch, iForest, SVM, LOF, and KNN. For RQ2, the focus shifts to the RCA algorithms, specifically MicroRCA, CausalRCA, CIRCA, and RCD. In addition, the co-independent variables are outlined in Table 4, where each variable value is considered a treatment for the experiment. While the order of treatments is randomized to avoid sequence bias, the specific treatment combinations are predefined to ensure that all possible combinations are systematically tested.

The usage scenario variable refers to simulated user interactions with the systems. These usage scenarios are described in Section 6.1. Two distinct user loads, 100 and 1000, simulate the number of concurrent users. Users are gradually added at a rate of 5 per second, taking 20 s to reach 100 users and 200 s to reach 1000 users, as illustrated in Fig. 6. This controlled ramp-up prevents the system from being overwhelmed, ensuring requests are handled more predictably. After each user fully executes the programmed jobs from the scenarios, the process starts again with newly generated users.

To focus on detecting and diagnosing energy consumption issues, anomalies are deliberately introduced by stressing the CPU, memory, and disk within a service. Specifically:

- The CPU is stressed by applying a computational workload on the CPU resources of the service container with a target load of 90% to increase the CPU usage significantly.
- Memory is stressed by allocating and accessing 921 MB of memory using one virtual memory worker; according to Docker Docs,[9] it is possible for a system's services to use the host machine's memory, so it is important to ensure that there is enough memory to perform important system functions; therefore, 921 MB of memory allocation is chosen in order to stress the system's memory while preventing system crashes.
- The disk is stressed by continuously and sequentially writing, reading and removing temporary files to the file system for a total of 921 MB.

It is important to note that we always stress CPU, memory, and disk *at the same time*; we do not have situations where we stress a service exclusively either the CPU or memory or the disk. Also, even though we stress the disk resource, we do not measure it when answering the RQs of the study.

Stress is applied to individual services rather than the entire system to establish a clear ground truth for evaluating the RCA algorithms. This stressed service can be one of two services per system. The services selected for the Sock Shop system are the front-end and orders services, which are the most relevant services in the two usage scenarios defined in Section 6.1. On the other hand, for the Train Ticket system, the stressed services were chosen based on their high dependency levels, corresponding to the number of incoming network requests in Fig. 5. The reasoning behind this is that when a service with many dependent services is stressed, it increases the likelihood of propagating the effects throughout the system. This is a problem proposed in this study, as it aims to evaluate the ability of RCA algorithms to accurately trace the root cause of anomalies in interconnected services, thus propagating effects could affect their performance. Therefore, the stressed services in the Train Ticket system are ts-order-service and ts-travel-service, which have 8 and 6 services, respectively, depending on them for network requests.

The normal operations treatment does not stress any services, thus simulating the systems' normal behavior with the different scenarios and user loads. Therefore, the data collected during normal behavior serves as a ground truth for evaluating the AD algorithms. Anomalies are defined using the three-sigma rule (Hochenbaum et al., 2017), so an energy consumption measurement in a service is considered anomalous

[9] https://docs.docker.com/engine/containers/resource_constraints/.

if it is more than 3 standard deviations away from the mean of the normal dataset.

This study aims to evaluate the effectiveness of different AD and RCA algorithms, so the dependent variables are the metrics used to answer the research questions. For RQ1, the Precision, Recall, and F-Score vary depending on the AD algorithm being evaluated and the combination of treatments.

For RQ2, the dependent variables are the precision at the top k (P@k) and the average precision at the top k (AP@k). The P@k score assesses the correctness of predictions within the top K root causes identified by the RCA algorithm. This metric is calculated according to Wu et al. (2021), by computing Eq. (1). In this equation, *R[i]* represents the rank assigned to a detected root cause, and *gt* symbolizes the ground truth, the actual, known root cause of the anomalies. In this experiment, *gt* is defined by the specific service where the stress was applied (*e.g.,* "orders service" in Sock Shop or "ts-travel-service" in Train Ticket). Moreover, *A* signifies the set of provided anomalies, including the whole anomalous dataset. Below is the equation used:

$$P@k = \frac{1}{|A|} \sum_{\alpha \in A} \frac{\sum_{1 \le i \le k} (R[i] \in gt)}{min(k, |gt|)} \tag{1}$$

Finally, the AP@k score calculates the overall performance of the RCA algorithms for each level k, where N is the number of top root causes considered. A top root cause refers to one of the k most likely root causes identified by an RCA algorithm, ranked by their relevance to the detected anomaly. These rankings are based on the algorithm's scoring or inference methods. Below is the corresponding formula:

$$AP@k = \frac{1}{N} \sum_{1 \le k \le N} P@k \tag{2}$$

5.4. Experiment design

The systems under test are deployed with the treatments listed in Table 4 using a cross-over design (Vegas et al., 2015). Every combination of user scenarios, stressed services, and user loads is applied to the target systems. The order of execution is randomized to remove any bias in the sequence in which these treatment combinations are executed. Additionally, this design ensures that all potential influences on the results, such as network and system conditions, are distributed across the runs. The metrics are also collected during the normal behavior of the systems, refraining from applying any stress. The remaining combinations of treatments are executed 30 times. This will serve as the data to train AD and RCA algorithms as well as the baseline to generate a ground truth for evaluating the AD results. Considering the 3 min warm-up period, the 2 min cooldown period, and the 5 min stress duration, and including the 12 possible treatment combinations and 30 repetitions, the total time required to replicate the experiment is approximately 60 h, or 2.5 days. This estimate closely matches the actual time taken to run our experiments.

6. Experiment execution

6.1. Subjects preparation

Locust.io[10] is used to create scripts that generate predictable traffic based on predefined usage scenarios and user loads. The usage scenarios in both Sock Shop and Train Ticket follow comparable patterns to ensure that meaningful comparisons between the systems can be made.

Sock Shop: Sock Shop's services were modified to incorporate stress-ng as done by Wu et al. (2020). The usage scenarios for Sock Shop are the same as those from Floroiu et al. (2024).

- Usage Scenario A: *Browse Catalogue* - Fig. 7 highlights the involved components.
  - The virtual user logs in, opens the catalogue, and randomly selects an item to preview a random number of times between 3 and 9.
  - The virtual user randomly waits between 1 and 3 s before performing each action.
- Usage Scenario B: *Create Order* - Fig. 8 highlights the involved components.
  - The virtual user logs in, opens the catalog, chooses between 3 and 9 items to add to the cart, and creates an order.
  - The virtual user randomly waits between 1 and 3 s before performing each action.

Train Ticket: Train Ticket did not need to be modified to incorporate stress-ng unlike Sock Shop

- Usage Scenario A: *Search Tickets* - Fig. 9 highlights the involved components.
  - The virtual user adds users to the database, logs in, and randomly searches for train tickets a random number of times between 3 and 9.
  - The virtual user randomly waits between 1 and 3 s before performing each action.
- Usage Scenario B: *Book Ticket* - Fig. 10 highlights the involved components.
  - The virtual user adds users to the database, logs in, searches for a train ticket, and books it.
  - The virtual user randomly waits between 1 and 3 s before performing each action.

In both systems, one scenario involves browsing available items (socks, train tickets), while the other usage scenario involves completing a transaction (placing an order, booking a ticket). Moreover, Scenario B is a continuation of Scenario A in both Sock Shop and Train Ticket, so Scenario B will always be more complex than Scenario A. In our setup, each microservice is deployed with a single replica. This configuration ensures that resource usage can be directly attributed to individual services, enabling more controlled and accurate measurements.

6.2. Setup

The systems of the experiment are deployed on two servers provided by Green Lab[11] which operate in parallel as illustrated in Fig. 11. These servers are configured with an Intel(R) Xeon(R) Silver 4208 CPU @ 2.10 GHz, 376 GB of RAM, and run on the Ubuntu 22.04.2 LTS operating system. Scaphandre[12] is utilized on these servers to measure power consumption values (in microwatts, μW) of the containers (services) in the host system. It uses the PowercapRAPL sensor, which provides insights into the total CPU consumption, consumption per CPU socket, and the consumption of the DRAM controller. This capability allows Scaphandre to compute power consumption for individual processes over a given timeframe. Moreover, another server is used to deploy Prometheus to collect performance-related metrics, with an Intel(R) Xeon(R) CPU E3-1231 v3 @ 3.40 GHz 8 vCPUs, 1TB of HD, and 32 GB of RAM, operating on the Ubuntu 22.04 LTS operating system. This data collection occurs on a dedicated server to ensure the measurements are unaffected by other factors. Finally, the data collection and data analysis programs are developed using Python version 3.10.11, and the systems are deployed using Docker version 25.0.3.

[10] https://locust.io/.

[11] https://s2group.cs.vu.nl/pages/greenlab/.

[12] https://hubblo-org.github.io/scaphandre-documentation/index.html.

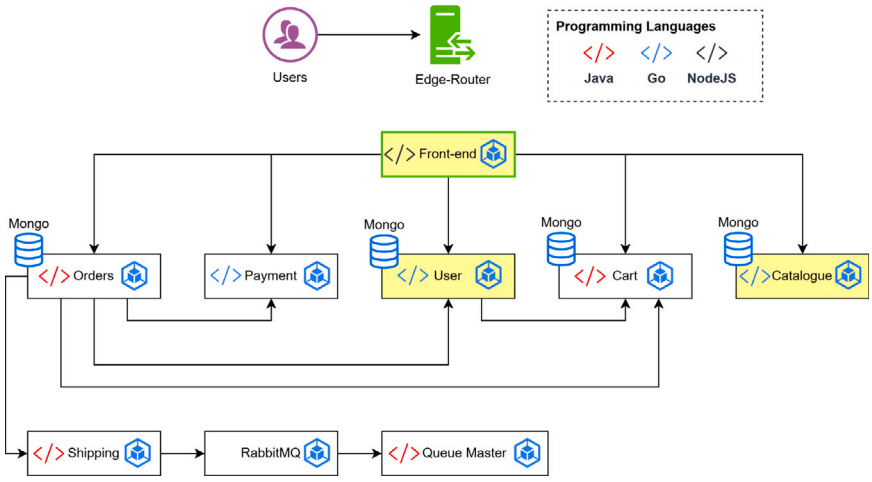


Fig. 7. Sock shop services involved in Scenario A.

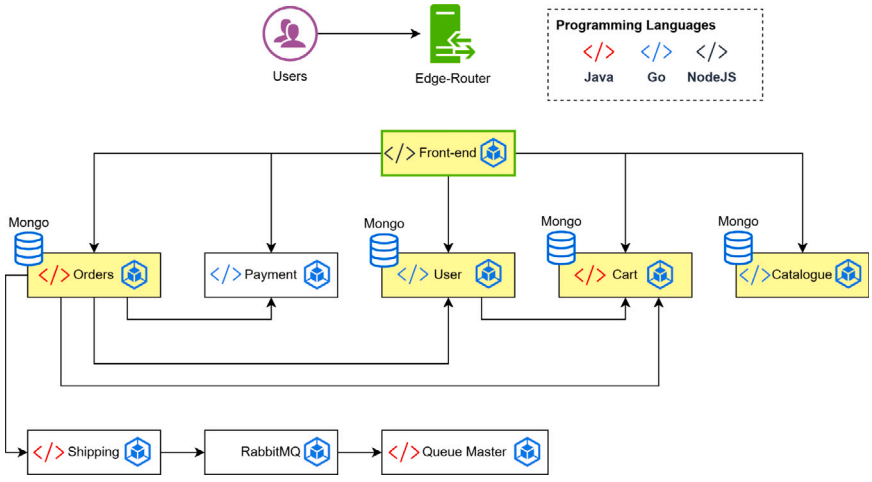


Fig. 8. Sock shop services involved in Scenario B.

6.3. Execution Phase 1: Controlled experiment and dataset construction

The experiment is intended to run each combination of treatments 30 times per system in order to generate enough data. Each repetition run follows the following steps:

1. Bring the system and monitoring services up
2. Wait for a warm-up period of 3 min to allow residual container resource consumption from the previous step to stabilize at idle levels
3. Randomly select treatment from pool, preventing order-related bias
4. Run user usage scenario via locust
5. Start service stress for a duration of 5 min
6. After 5 min, all processes are killed and data is collected from Prometheus
7. Wait for a cooldown period of 2 min to allow container resource consumption and machine CPU to stabilize to idle levels
8. Bring the system and monitoring services down to minimize their influence on the results
9. Repeat Steps 1–8 for the remaining treatment combinations

During an experiment run, a warm-up and cooldown period is injected to stabilize container resource usage around idle levels. These values are the same ones presented in the study by Floroiu et al. (2024). Adding time buffers in between experimental runs ensures that the data collected for each treatment is reliable and accurate. Moreover, anomalies are injected into individual services through the stress-ng[13] tool, which is a stress workload generator often used in research (Khan et al., 2018; Klaver et al., 2021). Regarding the monitoring services, Prometheus is used in combination with Scaphandre to collect different metrics. Prometheus collects data at 5-s intervals to gather enough instances without being too fine-grained. Scaphandre measures service-level power consumption metrics, which are later converted to energy consumption values.

6.3.1. Dataset construction

Individual service hardware resources and power consumption values are collected from Prometheus using PromQL[14] queries. Prometheus employs two monitoring tools: cAdvisor and node-exporter. Container Advisor (cAdvisor)[15] exposes resource usage and performance metrics at the service level, including CPU utilization, memory usage, file system activity, and network statistics. Node-exporter[16] measures system-level metrics from each host, allowing Prometheus to monitor from a separate server.

[13] https://wiki.ubuntu.com/Kernel/Reference/stress-ng.
[14] https://prometheus.io/docs/prometheus/latest/querying/basics/.
[15] https://prometheus.io/docs/guides/cadvisor/.
[16] https://prometheus.io/docs/guides/node-exporter/.

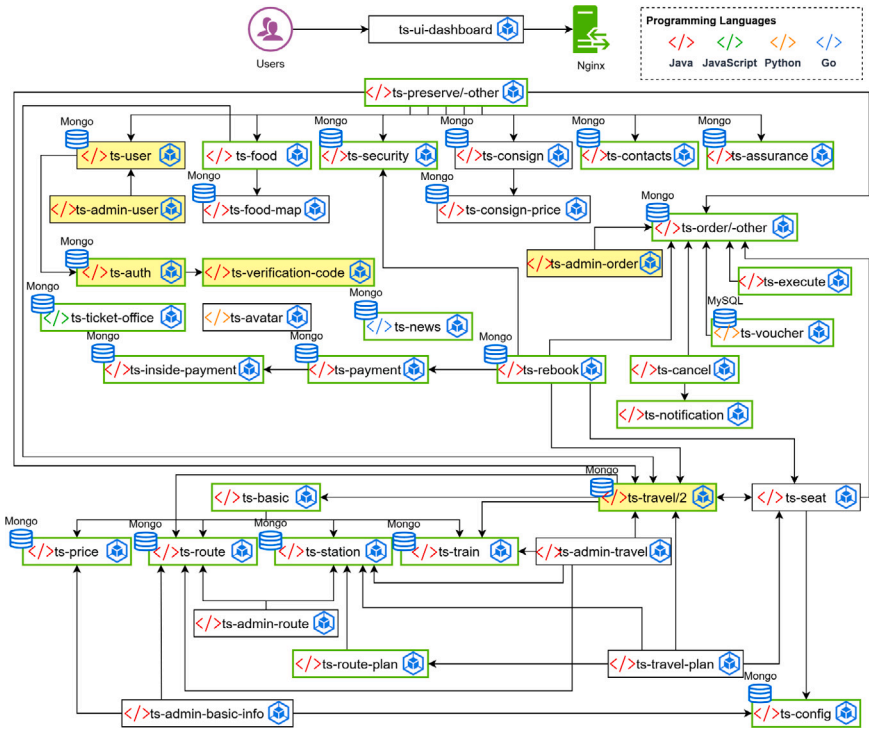


Fig. 9. Train ticket services involved in Scenario A.

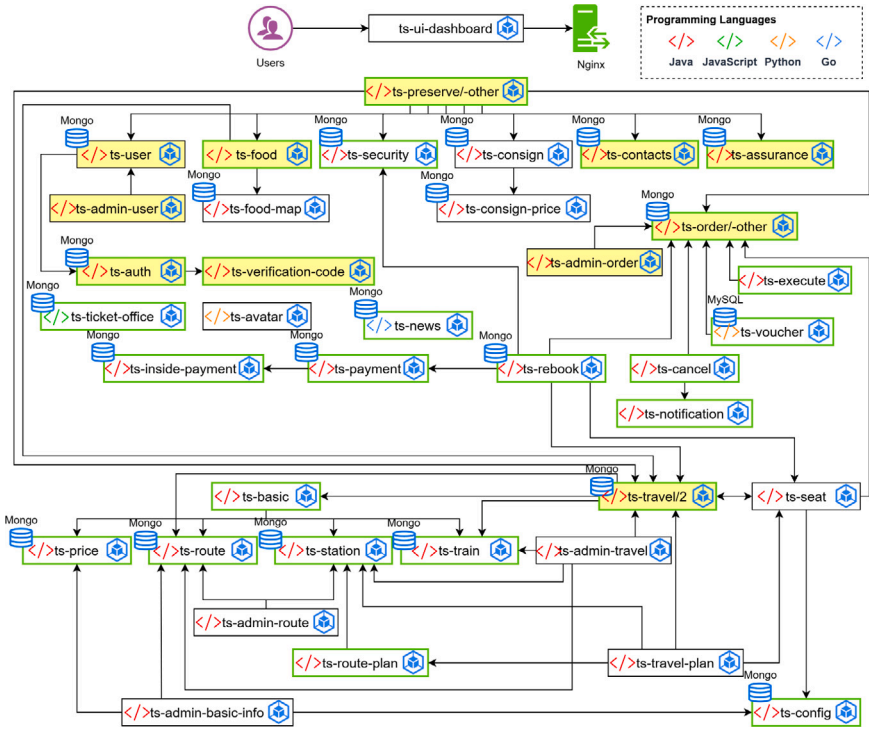


Fig. 10. Train ticket services involved in Scenario B.

After completing the experiment, the raw data collected is processed. Specifically, CPU usage values are converted into percentages by multiplying them by 100 for better readability. In addition, power consumption is converted to energy consumption in Joules by computing the Formula (3) so it can be used to answer the research questions of the study. P_{i-1} and P_i are the power consumption values in µW at consecutive points $i-1$ and i. Δt is the scrape interval in seconds, and 10^{-6} is the conversion factor from µW to W.

$$Energy(t) = \sum_{i=1}^{n} \frac{P_{i-1} + P_i}{2} \cdot \Delta t \cdot 10^{-6} \tag{3}$$

6.3.2. Dataset structure

Finally, the processed data is stored in two separate CSV files for each system: one for normal behavior data and another for anomalous

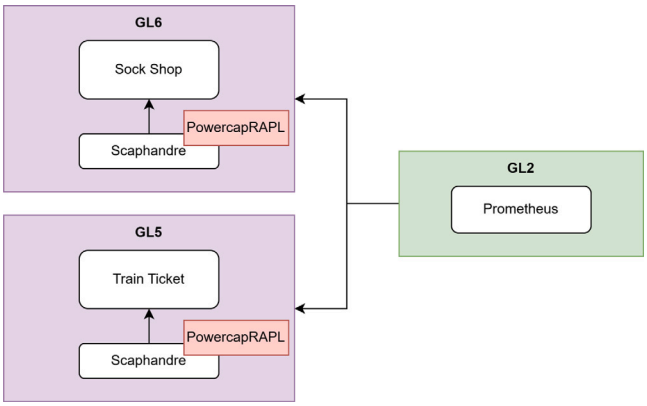


Fig. 11. Setup of the experiment.

data. These serve as the datasets used for the execution and analysis of the AD and RCA algorithms. Each dataset contains detailed metrics of performance and energy consumption of individual services over time. The performance metrics include CPU and memory usage for both the services and their associated databases. The energy consumption metrics, represent the power usage of each service and its database components over time. Additional columns include the usage scenario, stressed service, user load, and repetition number used for the duration of an experimental run. These details are helpful for analyzing the results based on the treatments selected.

6.4. Execution Phase 2: AD and RCA algorithms

6.4.1. Anomaly detection

The iForest, SVM, LOF, and KNN algorithms are executed using the PyCaret[17] library default configurations and methods. However, the Birch algorithm is executed based on the implementation used by Wu et al. in MicroRCA (Wu et al., 2020). The only changes made are the same ones as in the Floroiu et al. study (Floroiu et al., 2024), where the threshold for detecting an anomaly is changed so that only 5% of outliers are labeled as an anomaly. This adjustment is necessary because the data collected from the systems contains a higher proportion of normal instances than anomalous ones. By setting the threshold to label only 5% of data points as anomalies, the risk of overestimating anomalies is reduced, and it only requires changing one parameter.

6.4.2. Root cause analysis

The PyRCA[18] library is used to apply the RCD and CIRCA algorithms to the collected data. Therefore, the only configuration that needs to be specified is the number of top root causes to identify, which is k = 3. This value is chosen to provide a small, useful set of root causes for analysis, as done in other studies (Floroiu et al., 2024; Wu et al., 2020). For implementing the MicroRCA algorithm, several changes have been made to the original code: firstly, the BIRCH algorithm was modified as mentioned previously; secondly, all the code related to Kubernetes was removed as it does not apply to the current study; and finally, the metric (energy, CPU, or memory) of the identified root causes was added to the final result so it has the same format as other RCA algorithms. These customizations only required approximately 2 h, with most of the time spent on understanding the original implementation, and finding the right place to add the metric of the root causes.

[17] https://pycaret.gitbook.io/docs/get-started/quickstart.
[18] https://github.com/zhouruixingzhu/PyRCA.

6.4.3. Sensitivity analysis

For each AD and RCA algorithm, we systematically select a set of hyperparameters to assess the sensitivity of the results to different configurations. The selection process involves two researchers working independently; one researcher analyzes the documentation in the official repository of each AD/RCA algorithm, while the other researcher examines the scientific publication proposing the AD/RCA algorithm. Each researcher initially extracts the hyperparameters from one source (repository or paper) and then cross-checks the other source to verify each other's findings. The resulting lists of hyperparameters are merged, and only those hyperparameters that are explicitly documented in both the repository and the scientific publication are retained.

For each selected hyperparameter, we consider three values: the default value and two additional levels representing higher and lower configurations. When available, the hyperparameter levels were derived from empirical guidance or recommendations found in the scientific publication proposing the AD/RCA algorithm. In the absence of such references, the levels are defined based on commonly-adopted thresholds reported in the literature for the corresponding class of algorithms.

Table 5 shows the identified hyperparameters, along with the values selected for the low and high levels. We recall that for each hyperparameter, the choice of its values is informed by the corresponding tool's documentation and scientific publication. Specifically, the parameter levels for Isolation Forest, One-Class SVM, Local Outlier Factor, MicroRCA, and RCD were chosen based on those that produced acceptable and stable results in the respective experimental studies. For KNN, BIRCH, and CIRCA, where no clear guidelines are reported in both their documentation and scientific publication, we either selected the levels commonly-used in the libraries implementing them (*e.g.,* scikit-learn) or in guidelines provided by other tools using them.

In order to assess their sensitivity, we analyzed the performance of each algorithm across "low", "default", and "high" hyperparameter levels for both Sock Shop and Train Ticket, and under two user loads: 100 and 1000 users. As previously done, the performance is evaluated using (i) Precision, Recall, and F-Score for AD algorithms and (ii) AP@1, AP@2, and AP@3 for RCA algorithms.

7. Results

7.1. Dataset overview

7.1.1. Descriptive statistics

Table 6 provides the descriptive statistics of the metrics that were collected for the Sock Shop and Train Ticket systems. These include energy consumption, CPU usage, and memory usage of the entire system. In addition to overall memory usage, the Resident Set Size (RSS) usage represents the portion of memory actively used by all services, while memory cache usage is the portion used for caching purposes.

It can be observed from Table 6 that energy consumption in Train Ticket is significantly higher than in Sock Shop, with a mean of 28.837 J compared to Sock Shop's 7.813 J. This is to be expected considering that Train Ticket has more complex functionality and a greater number of active services than Sock Shop. This is also the case for CPU usage, where the maximum value for Train Ticket is 175.577% CPU usage, indicating that the services sometimes needed to use multiple CPU cores to perform the tasks. For Sock Shop, the memory usage ranges from 107.106 MB to 629.931 MB, with a mean of 349.148 MB. The Train Ticket system uses more memory in general, up to 925.656 MB with a mean of 479.618 MB. Sock Shop allocates more memory for caching than Train Ticket with a mean of 59.942 MB, meaning that it has faster data retrieval. In contrast, Train Ticket uses little cache, with a mean of 10.559 MB and instead relies more on active memory (RSS) for real-time data handling, with a mean of 244.078 MB. Finally, Sock Shop tends to show higher variability than Train Ticket, suggesting that the effect of the treatments chosen for the usage scenario, the service stressed, and the user load vary more in Sock Shop than in Train Ticket.

Table 5
Hyperparameter levels used for sensitivity analysis across AD and RCA algorithms.

Algorithm	Hyperparameter	Description	Levels
Isolation Forests	n_estimators	Number of trees in the forest	Default: 100, Low: 25, High: 200
One-Class SVM	ν	Upper bound on the fraction of training errors and, Lower bound of the fraction of support vectors	Default: 0.5, Low: 0.3, High: 0.7
Local Outlier Factor	n_neighbors	Number of neighbors to use by default for k neighbors queries	Default: 20, Low: 10, High: 35
KNN	n_neighbors	Number of neighbors to use by default for k neighbors queries	Default: 5, Low: 2, High: 10
BIRCH	threshold	Subcluster merging threshold	Default: 0.045, Low: 0.02, High: 0.065
MicroRCA	alpha	Fixed weight for edges flagged as anomalous when constructing the anomalous subgraph	Default: 0.55, Low: 0.3, High: 0.7
RCD	gamma	Subset size for variable partitioning in hierarchical learning	Default: 5, Low: 4, High: 6
CausalRCA	–	–	–
CIRCA	alpha	Desired significance level	Default: 0.05, Low: 0.01, High: 0.1

Table 6
Descriptive statistics of collected metrics (SD = standard deviation, CD = coefficient of variation).

Variable	Min.	Max.	Median	Mean	SD	CV
Sock Shop						
Energy (J)	0.094	33.442	5.766	7.813	5.500	70.393
CPU usage (%)	1.159	98.816	26.273	32.452	19.286	59.429
Memory usage (Mb)	107.106	629.931	351.327	349.148	65.312	18.706
Memory RSS usage (Mb)	6.868	379.910	134.708	136.125	55.037	40.431
Memory cache usage (Mb)	0.000136	183.992	62.824	59.942	22.745	37.946
Train Ticket						
Energy (J)	1.657	70.321	26.983	28.837	13.787	47.811
CPU usage (%)	7.574	175.577	87.263	88.934	34.351	38.626
Memory usage (Mb)	106.853	925.656	489.143	479.618	139.829	29.154
Memory RSS usage (Mb)	9.758	593.535	242.228	244.078	89.025	36.474
Memory cache usage (Mb)	0.224	45.948	8.973	10.559	7.871	74.548

7.1.2. Anomalous vs. normal behavior

Fig. 12 shows a comparison of the energy consumption metrics per service between normal and anomalous behavior in both Sock Shop and Train Ticket. This serves as a way to make sure that the deliberate introduction of anomalies by stressing specific services works as intended. As seen in Fig. 12(a), the services that show the largest difference between the normal and anomalous data are the front-end and orders services, and in Fig. 12(b), the same happens with ts-order-service and ts-travel-service. These are precisely the services that have been stressed in Sock Shop and Train Ticket, respectively (See Table 4), meaning that the stressing works as expected.

Moreover, the Sock Shop system shows more stable energy consumption across its services than the Train Ticket system, possibly due to lighter workload. In fact, the distribution of some services such as the payment service is concentrated near zero, showing minimal energy consumption. In order to improve the readability of the Train Ticket plots, only the top 12 services with the highest distribution are included. Among these Train Ticket services, they tend to consume more energy than those in Sock Shop due to their more complex operations. The ts-order-service has many outliers in both the normal and anomalous data, meaning that there were instances where the service consumed significantly more energy than usual. These outliers suggest possible inefficiencies in how the service handles high workloads. Notably, the ts-auth-service has the highest median energy consumption among all services in both normal and anomalous behavior, even higher than the stressed services. This shows its crucial role in the system, as it is in charge of authenticating any user. As the number of users in Train Ticket increases, it consumes more energy.

7.1.3. Scenario

Fig. 13 provides a comparison of the energy consumption metrics of each service between the usage scenarios described in Section 6.1 for Sock Shop and Train Ticket. In Sock Shop, the services carts and carts-db have higher energy consumption for the Create Order scenario than for the Browse Catalogue scenario. This is because in the Create Order scenario, the items are added to the cart, whereas in Browse Catalogue items are simply viewed. In Train Ticket, the services involved in the searching process show higher energy consumption for the Search Tickets scenario, such as ts-route-service, ts-station-service, ts-ticketinfo-service, and ts-travel(2)-service. On the other hand, ts-order-service is the most important service in the Book Ticket scenario, so further stressing it causes the energy consumption to increase considerably. Furthermore, in ts-auth-service the energy consumption is higher for the Book Ticket scenario than for the Search Ticket scenario because also checks whether the user is logged in before allowing them to start the booking process.

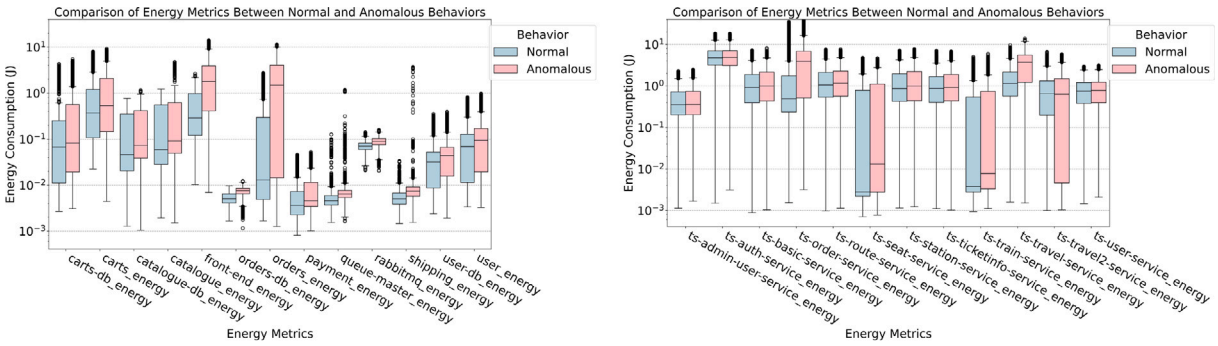
7.1.4. Stressed service

Fig. 14 illustrates the comparison of the energy consumption metrics of each service between the stressed services, which further proves that the selected services were correctly stressed. In Sock Shop, the front-end service has the highest energy consumption when stressed, while the orders service takes this position when it becomes the stressed service. Similarly, in Train Ticket, when the ts-order-service or the ts-travel-service is stressed, their median energy consumption is slightly higher than ts-auth-service.

It is also important to investigate whether stressing the selected services causes a propagation effect. In both Sock Shop and Train Ticket, no other services show significant differences in energy consumption when one service is stressed compared to another, likely due to the independence between services. Therefore, stressing the selected services does not seem to cause any propagation effect on the energy consumption of other services. This implies that for the ground truth of the RCA algorithms, the root cause of the anomalies should only involve the metrics related to the stressed services, and no other services.

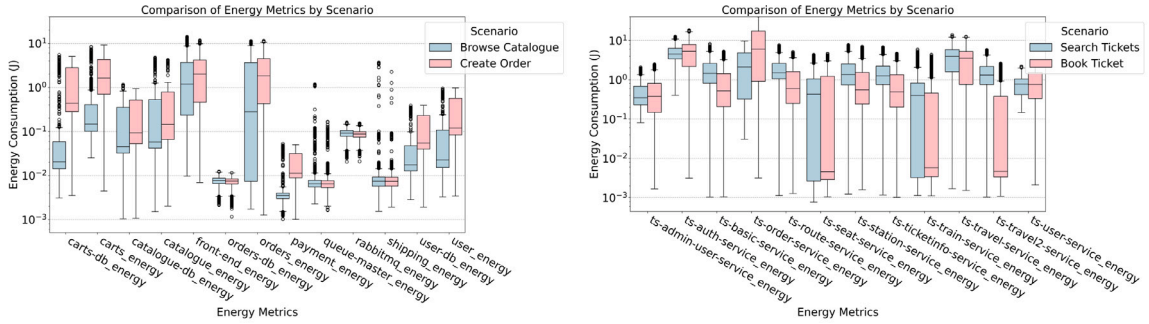
7.1.5. User load

Fig. 15 compares the energy consumption of each service based on the user load for both Sock Shop and Train Ticket. In both systems, having 1000 concurrent users increases the energy consumption of all services. This is expected, as more concurrent users result in more network requests and increased resource utilization. In Sock Shop, the increase in energy consumption between 100 and 1000 users is quite noticeable across most services. However, in Train Ticket the difference in energy consumption between the two user loads is less



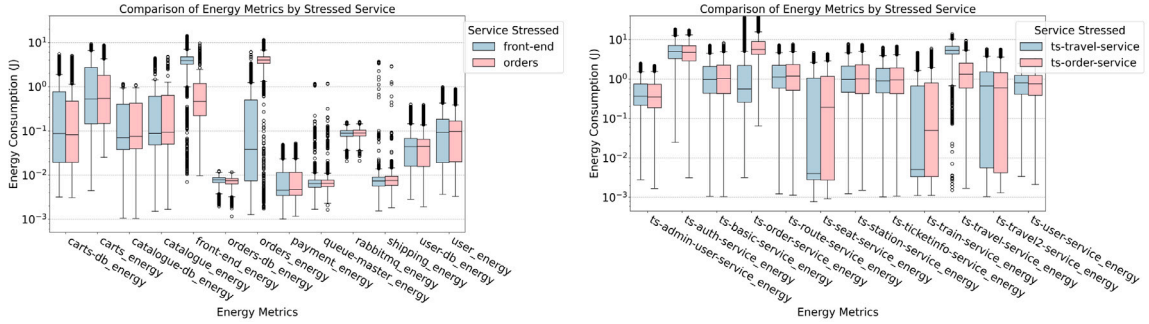
(a) *Sock Shop* (b) *Train Ticket*

Fig. 12. Distribution of energy metrics (Normal vs. Anomalous behavior).



(a) *Sock Shop* (b) *Train Ticket*

Fig. 13. Distribution of energy metrics by scenario.



(a) *Sock Shop* (b) *Train Ticket*

Fig. 14. Distribution of energy metrics by stressed service.

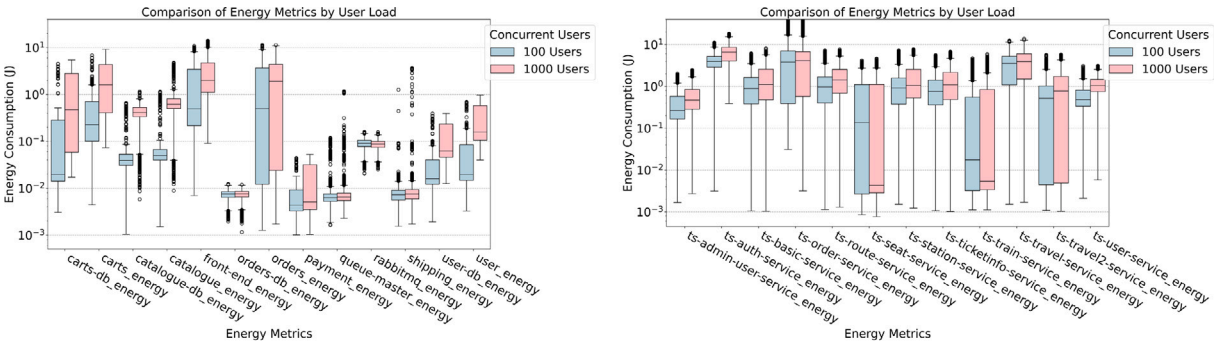
obvious, except for ts-auth-service, which handles user authentication, and managing a larger number of concurrent users increases its energy consumption. Therefore, the user load had a greater effect in the energy consumption of Sock Shop than Train Ticket.

7.2. RQ1: Effectiveness of AD algorithms in detecting energy issues

7.2.1. Precision

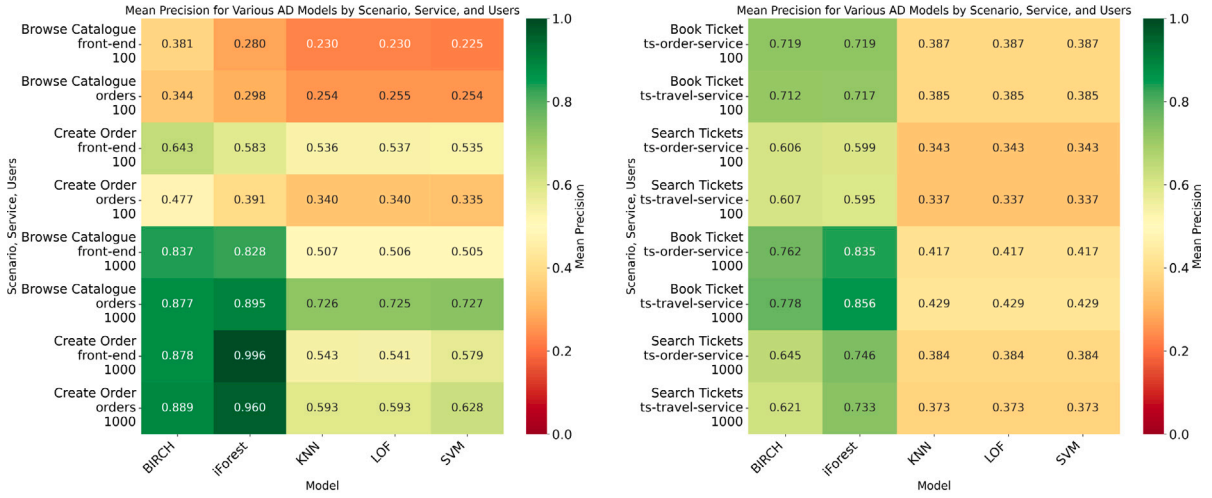
As seen in Fig. 16, in general, the BIRCH and iForest algorithms perform the best for both the Sock Shop and Train Ticket systems, indicating their ability to correctly detect energy anomalies across both datasets. In Sock Shop, the overall mean precision for BIRCH is 0.861, which is higher than iForest's, 0.758, whereas in Train Ticket iForest outperforms BIRCH with a value of 0.725. The reason behind their relatively good performance in Sock Shop as well as a larger system like Train Ticket could be due to their ability to scale well with large datasets, which is one of their common characteristics in Table 2. In contrast, KNN, LOF, and SVM generally have lower precision when detecting anomalies. Looking at their common characteristics in Table 2, the reason for their poor performance could be their sensitiveness to the parameter chosen. KNN, LOF, and SVM rely heavily on hyperparameters like the number of neighbors, the neighborhood density threshold, and the kernel type, respectively. These can lead to overfitting, where normal instances are misclassified as anomalies. Hyperparameter tuning is a possible approach to improve the performance of these algorithms (see Section 7.4).

Additionally, Fig. 16 presents the mean precision values of the AD algorithms based on the scenario, the stressed service, and the user load treatments described in Section 5.3. It can be observed that in Sock Shop, all AD algorithms tend to perform better when the user load is 1000. This is likely because, as discussed in Section 7.1.5, higher user loads tend to increase resource demands and energy consumption, making anomalies more noticeable. In the Sock Shop dataset, when



(a) *Sock Shop* (b) *Train Ticket*

Fig. 15. Distribution of energy metrics by user load.



(a) *Sock Shop* (b) *Train Ticket*

Fig. 16. Mean precision.

the usage scenario Browse Catalogue is applied, the orders service is stressed, and there are 1000 concurrent users, all the algorithms consistently perform well. As for the Train Ticket dataset, the best treatment combination is when the usage scenario Book Ticket is applied, the ts-travel-service is stressed, and there are 1000 concurrent users. This happens to be one of the most complex treatment combinations, which may have caused the energy anomaly to become more apparent.

Finally, only in Sock Shop we observe a case where the stressed service seems makes a difference in the precision of the algorithms. The treatment combination where the front-end service is stressed, the Create Order scenario is used, and there are 100 users, has a higher precision across models than when the orders service is stressed with the same usage scenario and user load. This could be due to the front-end service being directly involved in user interactions, making its resource anomalies more easily detectable by the AD algorithms. In contrast, the orders service operates more in the background, where resource usage changes might be subtler.

7.2.2. Recall

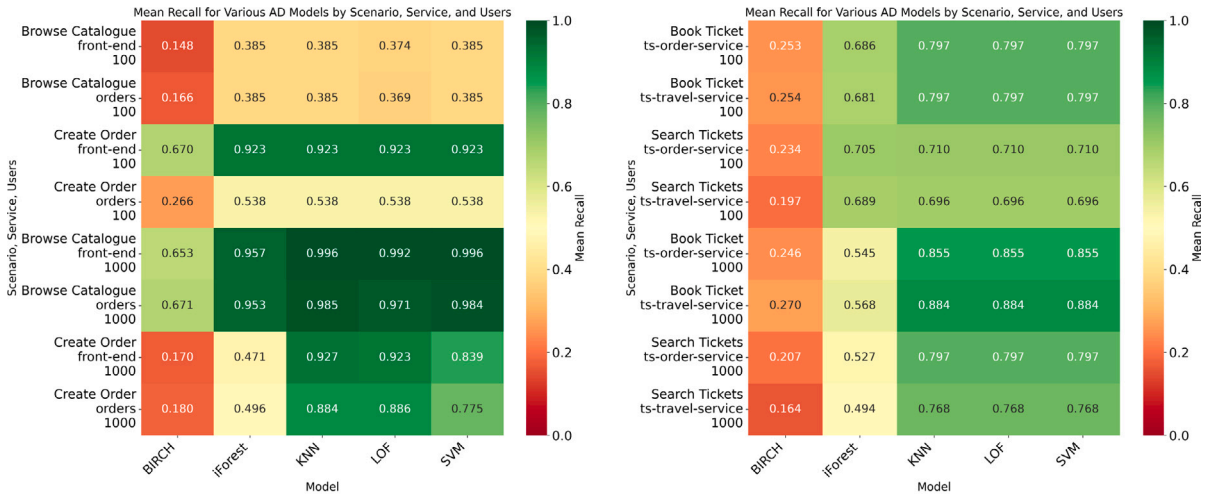
Overall, the recall values in Fig. 17 show a different pattern compared to precision. While BIRCH performs well in terms of precision, it struggles with recall in both the Sock Shop and the Train Ticket datasets, *i.e.,* it often misses true energy anomalies. When looking at BIRCH's characteristics in Table 2, this is likely due to its clustering method. Nevertheless, the other models do well in detecting many of the true anomalies, especially the KNN, LOF, and SVM algorithms. The sensitiveness that affected them when it came to precision may have been the reason for their good performance now.

Fig. 17 also shows the mean recall values of the AD algorithms according to the scenario, the service stressed, and the user load treatments. In Sock Shop, the best recall values are found with the Browse Catalogue scenario, the front-end service stressed, and 1000 users. This treatment combination only differs from the best precision result by the service stressed. Moreover, the difference between stressing the front-end service and the orders service in Sock Shop is more apparent in recall when the Create Order scenario is used and there are 100 concurrent users. This means that stressing the front-end slightly improves the completeness of the algorithm compared to stressing the orders service.

In Train Ticket, the best recall values are achieved under the same treatments as in precision: with the Book Ticket scenario, ts-travel-service stressed and 1000 users. However, although KNN, LOF, and SVM generally have better recall with 1000 users, iForest performs better in recall when the user load is 100. In Section 7.1.5, it was observed that the energy consumption of Train Ticket is less sensitive user load compared to Sock Shop. Therefore, the effectiveness of AD algorithms in recall may not follow a straightforward pattern like in Sock Shop.

7.2.3. F-Score

The F-Score provides a good indicator of which is the overall best AD algorithm and treatment combination for both systems since it combines the precision and recall metrics. According to Fig. 18, in both the



(a) *Sock Shop* (b) *Train Ticket*

Fig. 17. Mean recall.

Sock Shop and Train Ticket datasets, iForest has the best F-score values in most treatment combinations. In Sock Shop, iForest's mean F-score is 0.590, compared to KNN's 0.570. In Train Ticket, iForest's mean F-score is 0.634, compared to KNN's 0.514. This means that iForest is the most consistent and reliable at detecting energy consumption anomalies, although there is still a lot of room for improvement. Its scalability and isolation methodology could account for its performance throughout the treatments and datasets. The BIRCH algorithm's overall performance is affected by its poor recall values, whereas KNN, LOF, and SVM compensate their poor precision values with their recall values. In the end, these three AD algorithms outperform BIRCH even though BIRCH has higher precision values.

The optimal treatment combination for Sock Shop is when the Browse Catalogue is applied, the orders service is stressed, and 1000 users are using the system. For Train Ticket, as in precision and recall, the best results occur with the Create Order scenario, the ts-travel-service stressed, and 1000 users. In Sock Shop, stressing the front-end service under the Create Order scenario and 100 users results in a higher F-score in all models than stressing the orders service under the same conditions.

Looking at all the AD results, it turns out that most of the algorithms perform well either in precision or recall, with the exception of iForest, which exhibits values greater than 0.5 in both precision and recall in the vast majority of the tested configurations. Importantly, performance is particularly good in the most complex and realistic setting, that is Train Ticket, especially with many users. Achieving a precision, recall and F-score above 0.5 in AD makes the above algorithms indeed worth to use, as performance is significantly better than random guessing in imbalanced datasets (which is common in anomaly detection, where anomalies are rare). For example, if anomalies represent only 5% of the data, a classifier that naively guesses will likely perform poorly. Achieving scores over 0.5 shows the algorithm learns real patterns. Moreover, as anomalies can cause cascading failures in microservices, and even partially successful detection (F-score > 0.5) is often enough to trigger meaningful signal, prevent major incidents, and reduce the Mean Time to Recovery (MTTR) — particularly important when the cost of missing an anomaly is high (e.g., downtime, SLA violations). Fine-tuning in a specific setting as well as complementing the algorithms with existing rule-based alerts or SLO-based monitors can, of course, further improve performance.

RQ1: Effectiveness of AD algorithms in detecting energy issues

Overall, results show that the tested algorithms are good candidates for energy-related AD. The best-performing AD algorithm is **iForest**, as it achieves the highest F-score across most treatment combinations (0.590 in Sock Shop and 0.634 in Train Ticket), making it the most consistent and reliable technique for detecting energy anomalies. **BIRCH** performs well in terms of precision, especially in Sock Shop, but its low recall affects its overall effectiveness. On the other hand, **KNN, LOF, and SVM** have strong recall, but struggle with precision since they require hyperparameters fine-tuning to improve the overall performance.

7.3. RQ2: Effectiveness of RCA algorithms in identifying services being the root cause of energy issues

7.3.1. Average precision @1 (AP@1)

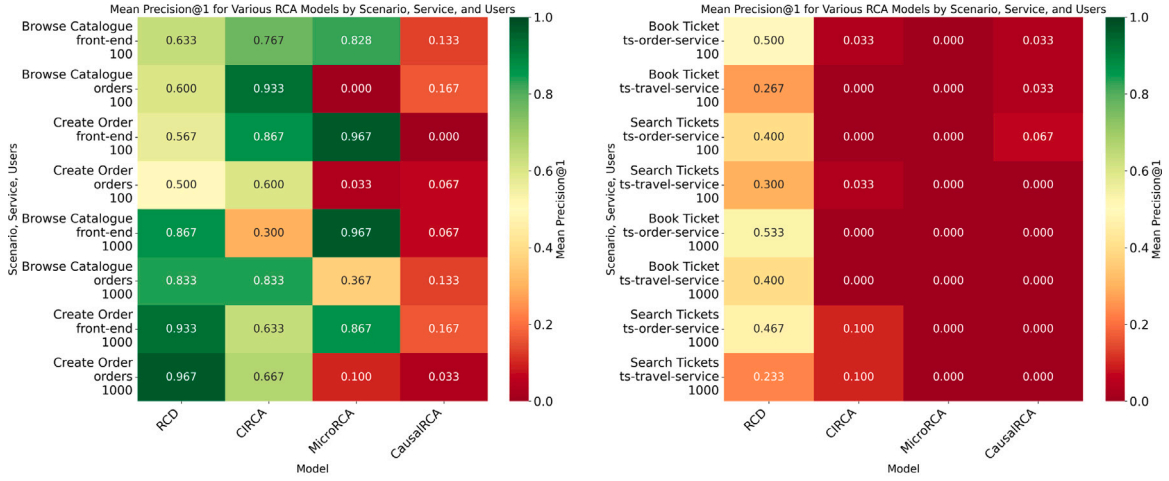
Fig. 19 shows the mean precision of the top-identified root cause (AP@1) for the RCA algorithms according to the scenario, the stressed service, and the users load treatments. Overall, the RCA results are worse than the AD results. Very few combinations of RCA algorithms and applied treatments exhibit high precision in identifying the correct top root cause of energy anomalies.

In Sock Shop (see Fig. 19(a)), RCD outperforms the other algorithms. This result is likely due to RCD's lightweight architecture and the utilization of scalable causal discovery techniques, such as hierarchical learning and conditional independence tests (Ikram et al., 2022). These methods allow RCD to efficiently analyze large datasets by limiting the complexity of constructing and evaluating the causal graph. Moreover, the treatment combinations with 1000 users exhibits a high AP@1 (at least 0.833). This further proves that having higher user loads can help the performance of AD and RCA algorithms, as it increases the performance and energy metrics. The algorithm with the next highest AP@1 in Sock Shop is CIRCA. This can be attributed to CIRCA's advanced methodology, such as regression-based hypothesis testing and descendant adjustment, as described in Table 3. Differently, MicroRCA has a very high AP@1 only when the front-end service is stressed. This is likely due to the fact that, as shown in Table 3, it uses a topology graph to do its analysis, so how relevant a service is in a system can impact how well it finds the root causes. As discussed in the previous section, the front-end service is more involved in the system,



(a) *Sock Shop* (b) *Train Ticket*

Fig. 18. Mean F-Score.



(a) *Sock Shop* (b) *Train Ticket*

Fig. 19. AP@1 of all RCA algorithms on SockShop and train ticket.

whereas the orders service works more in the background. CausalRCA has a very low AP@1 across all combinations of treatments in both the SockShop and the Train Ticket systems.

The AP@1 of all RCA algorithms significantly drops when considering the Train Ticket dataset (see Fig. 19(b)), which hints that those algorithms are unable to scale to a larger system. We speculate that the reason behind the RCA algorithms not performing well with Train Ticket could be because its topology and causal graph is significantly larger and more complex, making it more difficult to trace the anomalies back to their root cause.

7.3.2. Average precision @2 (AP@2)

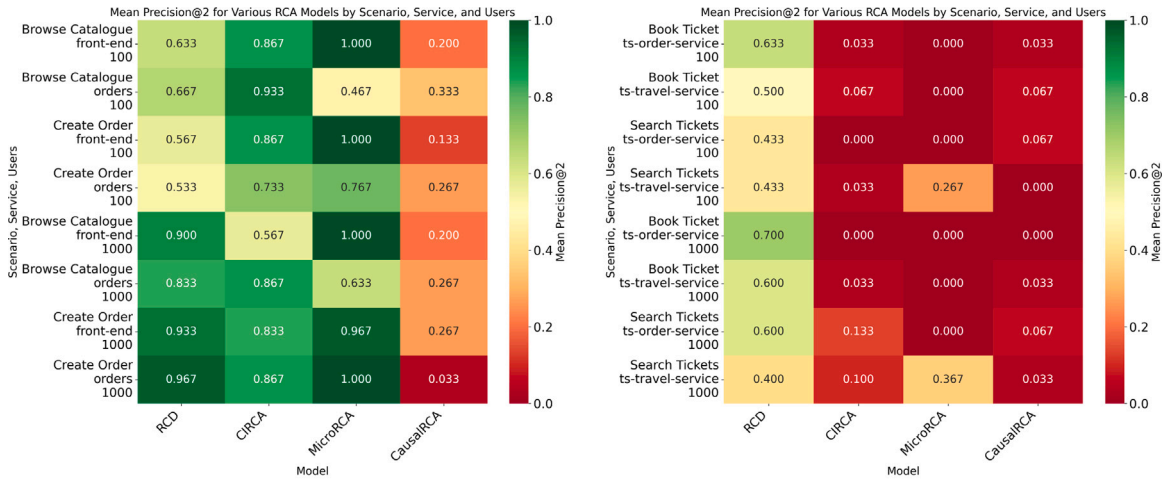
Fig. 20 shows the average precision of the top-2 identified root causes of the RCA algorithms according to the scenario, the stressed service, and the user load treatments. The performance of the algorithms increases across all models for both systems.

In Sock Shop (see Fig. 20(a)), the AP@2 of RCD remains very high, while the CIRCA and MicroRCA algorithms improve considerably. For several treatment combinations, MicroRCA now even outperforms the CIRCA and RCD algorithms — which were the best when it came to AP@1. CausalRCA still remains the algorithm with the worst AP@2. Unfortunately, there is almost no change in the performance of the algorithms when it comes to the Train Ticket system (see Fig. 20(b)). However, RCD is the most reliable across systems, as it has high AP@2 scores in Sock Shop and moderate AP@2 scores in Train Ticket.

7.3.3. Average precision @3 (AP@3)

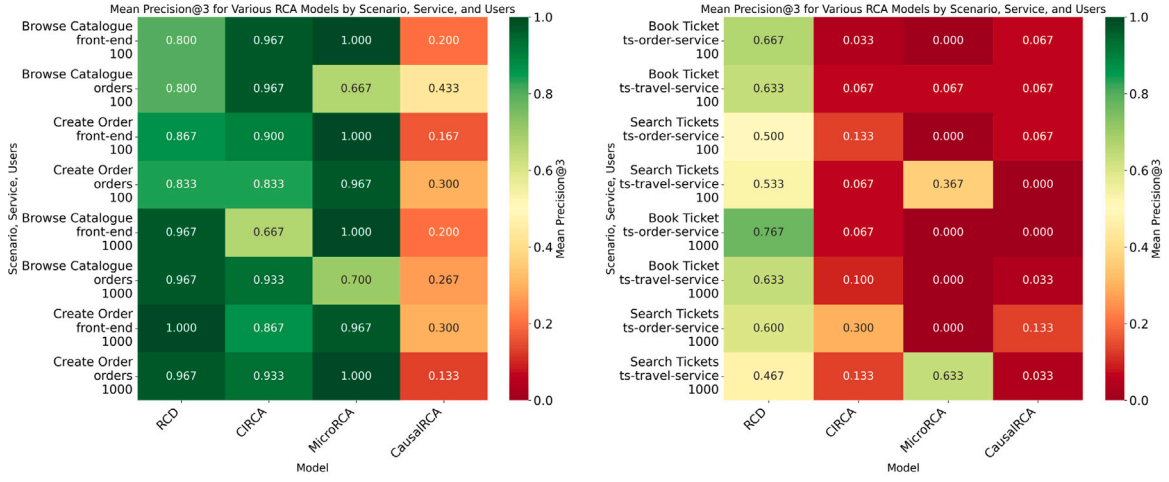
Fig. 21 shows the mean precision of the top-3 identified root causes of the RCA algorithms according to the scenario, the stressed service, and the user load treatments. The results for AP@3 follow the same trends as those of AP@2, with the only noticeable difference that the values of AP@3 are sensibly higher than those of AP@3 across all combinations of algorithms, treatments, and considered system.

Overall, looking at the above results, the tested RCA algorithms, unlike the AD ones, do not consistently exhibit acceptable performance, even when increasing k up to 3. No algorithm consistently exceeds a Precision@k of 0.5, except RCD starting from $k \geq 2$, meaning that, on average, only half of the relevant causes are found early enough to be useful. Compared to the precision of the same algorithms applied to performance metrics in their original papers (*e.g.,* PR@1 up to 0.89 in SockShop for MicroRCA), results obtained with energy consumption suggest the need to significantly improve current algorithms. With the obtained values, the application could not be justifiable, as the algorithm may not outperform manual inspection or simple heuristics



(a) *Sock Shop* (b) *Train Ticket*

Fig. 20. AP@2 of all RCA algorithms on SockShop and train ticket.



(a) *Sock Shop* (b) *Train Ticket*

Fig. 21. AP@3 of all RCA algorithms on SockShop and train ticket.

(*e.g.,* service dependencies, error spike correlation, or lightweight alert correlation analysis), potentially leading to remarkably longer MTTR and higher recovery costs. In addition, when non-relevant items frequently outrank the true root cause, engineers lose trust in the tool and its suggestion tend to be ignored.

RQ2: Effectiveness of RCA algorithms in identifying services being the root cause of energy issues

Compared to AD algorithms, RCA algorithms are less effective, showing significantly worse results. **RCD** consistently performs relatively well across both Sock Shop and Train Ticket, leveraging scalable causal discovery techniques to trace anomalies efficiently, even in large systems. **CIRCA** achieves high precision (even in terms of AP@1) in Sock Shop but struggles to scale to larger systems like Train Ticket, while **MicroRCA** occasionally performs better when considering AP@2 and AP@3.

7.4. Sensitivity analysis

As shown in Figs. 22 and 23, in terms of Precision for 100 and 1000 users, BIRCH exhibits low sensitivity to its *threshold* parameter in both systems, with the `low` level median being only slightly higher than the `default` value. Differently, iForest exhibits high sensitivity in both systems to its *n_estimators* parameter, with its `default` setting achieving a significantly higher Precision than both the `low` and the `high` values. KNN also demonstrates high sensitivity for both Sock Shop and Train Ticket, and across both user loads; the `low` and `high` values for the *n_neighbors* parameter are considerably lower than the `default` value. In the case of LOF, except for Sock Shop with 100 users, its `default` *n_neighbors* value exhibits a notably lower Precision than its `low` and `high` values. In both user load scenarios and systems, SVM's `default` value for ν consistently has better Precision than its alternative levels. A similar pattern occurs with Recall and F-Score in Figs. 24, 25, 26, and 27, with the exception of the Recall of LOF, where the default value of *n_neighbors* in both systems and user loads outperforms the `low` and `high` levels.

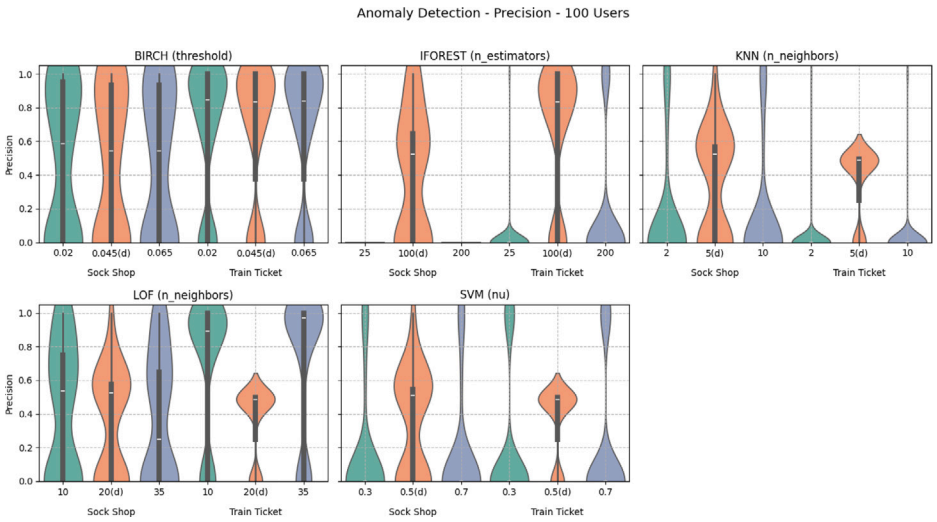


Fig. 22. Precision distribution for AD algorithms across parameter levels with a user load of 100.

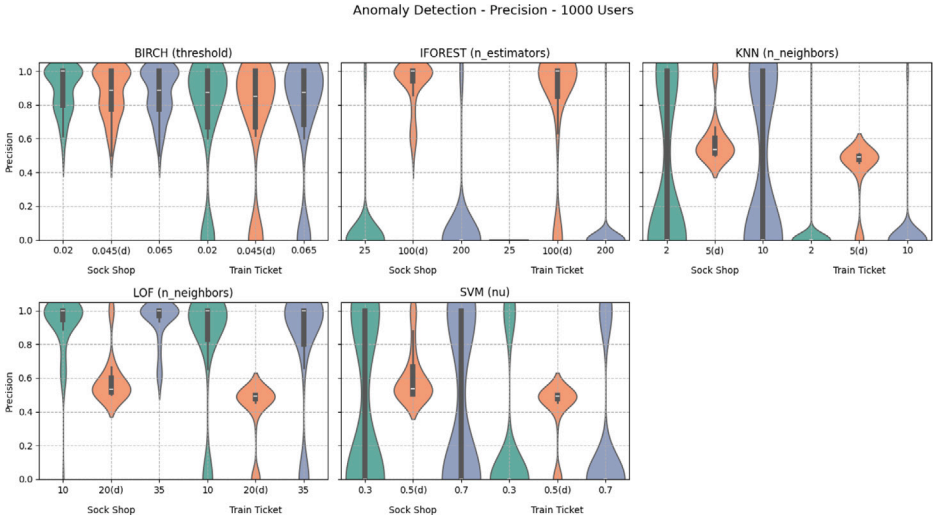


Fig. 23. Precision distribution for AD algorithms across hyperparameter levels with a user load of 1000.

In summary, the obtained results show that the default parameter settings tended to produce the best or at least comparable outcomes across most metrics and models. Nevertheless, there are isolated instances – particularly with the LOF algorithm – where alternative parameter values yield (marginal) improvements on specific metrics.

Figs. 28 and 29 show the results of PR@1 of the RCA algorithms considered in this study. CIRCA exhibits low sensitivity to changes in its *alpha* hyperparameter, with the same median for its `default`, `low`, and `high` values in both Sock Shop and Train Ticket systems and user loads. Differently, MicroRCA's `default` *alpha* value yields slightly worse results compared to alternative levels in both systems and user loads. RCD's *gamma* values follow the same trends as those of MicroRCA's *alpha* values; however, differently from MicroRCA, in Train Ticket the `default` value of RCD yields higher PR@1 results compared to the `low` and `high` levels in both user loads. Figs. 30 and 31 show the results for the PR@2 metric. Differently from the PR@1 results, the `default` values of all RCA algorithms consistently yield a higher PR@2 compared to the `low` and `high` levels across both Sock Shop and Train Ticket systems and the two user loads. In particular, CIRCA obtains a higher median but a similar multimodal distribution of results. Although MicroRCA and RCD default values obtain similar median values, these are the only levels that obtain the maximum PR@2. Finally, Figs. 32 and 33 show the results for PR@3. The differences between levels for each RCA algorithm are the same compared as those observed in the results for PR@2, but more pronounced.

In summary, for what concerns RCA algorithms, we can conclude that the `default` settings of all RCA algorithms tend to consistently have higher PR@2 and AP@3 than their alternative values; for what concerns PR@1, CIRCA exhibits low sensitivity, whereas MicroRCA and RCD are more sensitive with respect to their considered hyperparameters.

8. Discussion

The results of anomaly detection (AD) and root cause analysis (RCA) algorithms revealed several important insights into their effectiveness in detecting and diagnosing energy consumption issues in microservice-based systems.

For AD algorithms, iForest and BIRCH consistently showed higher precision, especially under high user loads, where anomalies were more apparent due to increased resource demands. Future research should investigate how sudden spikes in terms of the number of users impact the performance of AD and RCA algorithms. However, while BIRCH excelled in precision, it showed lower recall, indicating a tendency to miss true anomalies. This proves the trade-offs between detecting

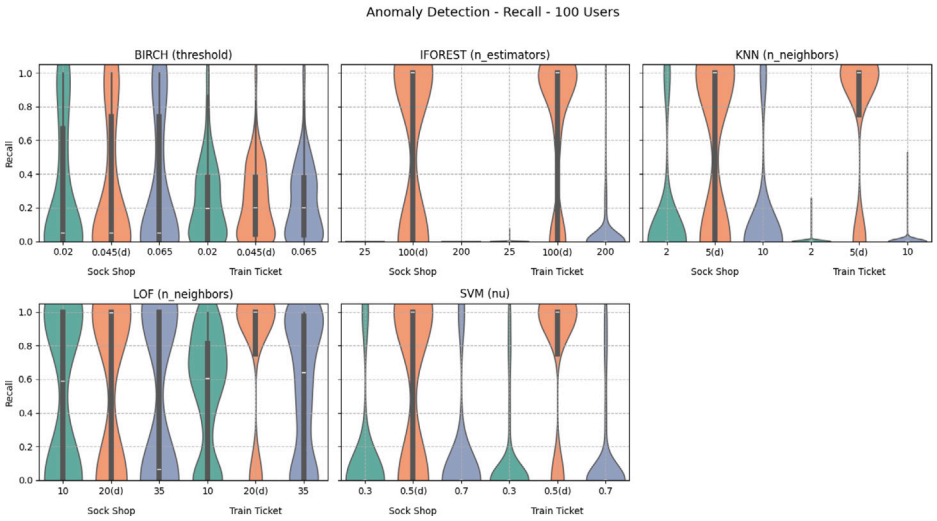


Fig. 24. Recall distribution for AD algorithms across hyperparameter levels with a user load of 100.

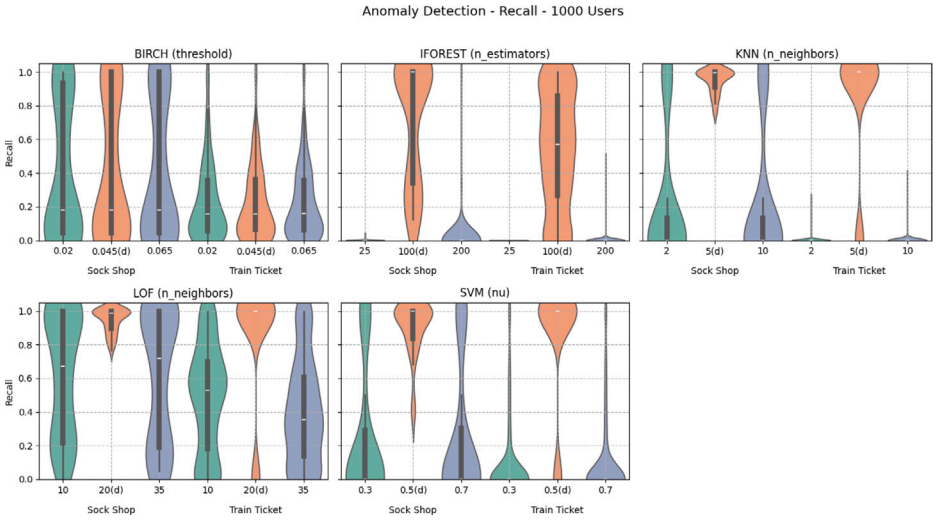


Fig. 25. Recall distribution for AD algorithms across hyperparameter levels with a user load of 1000.

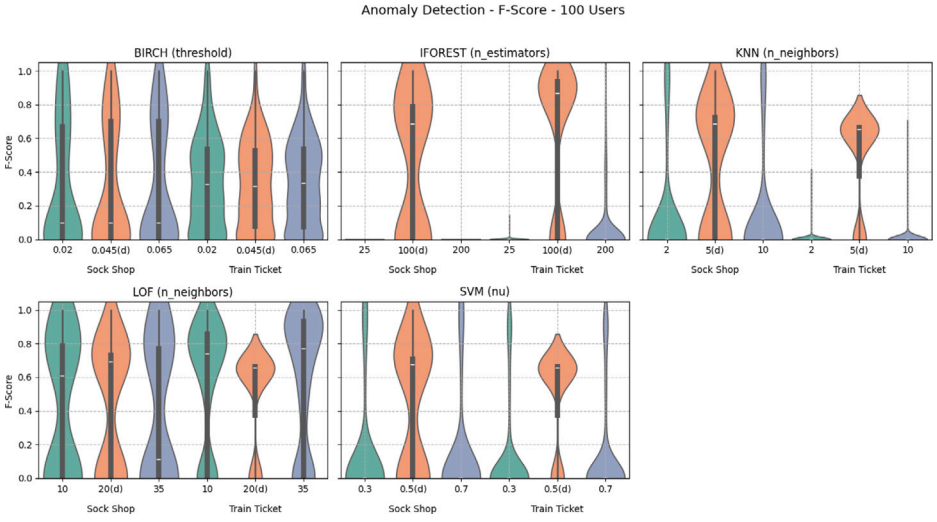


Fig. 26. F-Score distribution for AD algorithms across hyperparameter levels with a user load of 100.

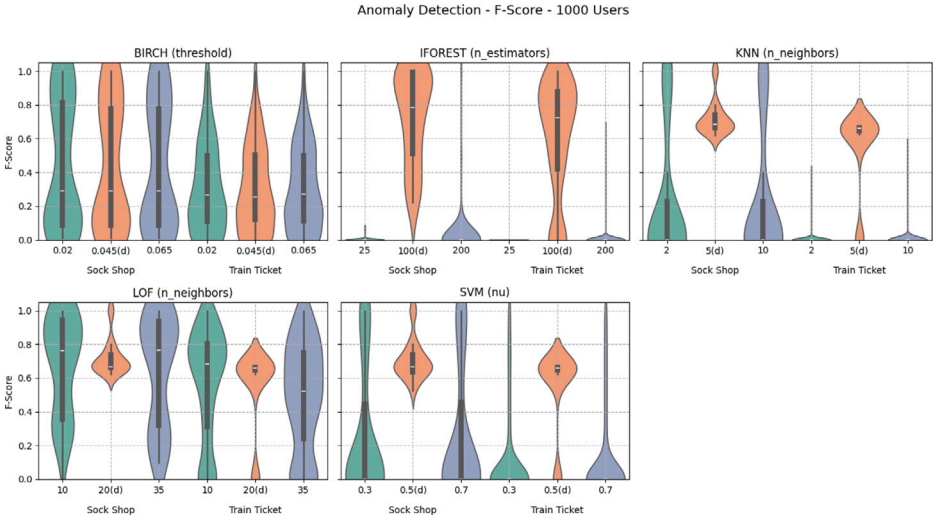


Fig. 27. F-Score distribution for AD algorithms across hyperparameter levels with a user load of 1000.

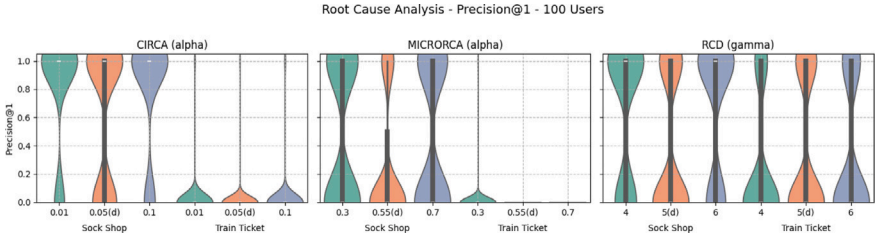


Fig. 28. PR@1 distribution for RCA algorithms across hyperparameter levels with a user load of 100.

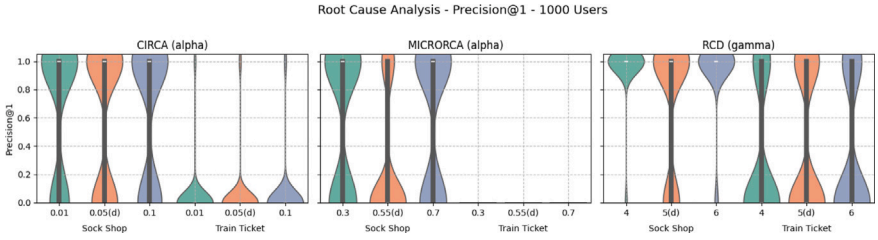


Fig. 29. PR@1 distribution for RCA algorithms across hyperparameter levels with a user load of 1000.

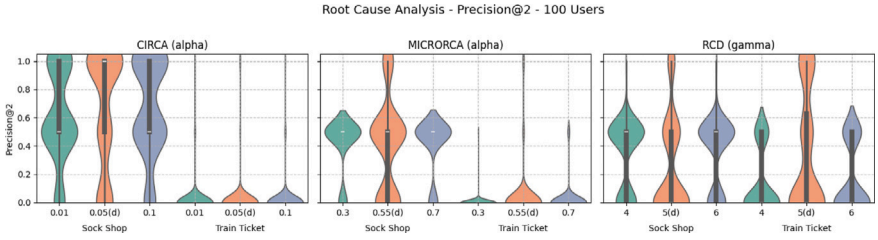


Fig. 30. PR@2 distribution for RCA algorithms across hyperparameter levels with a user load of 100.

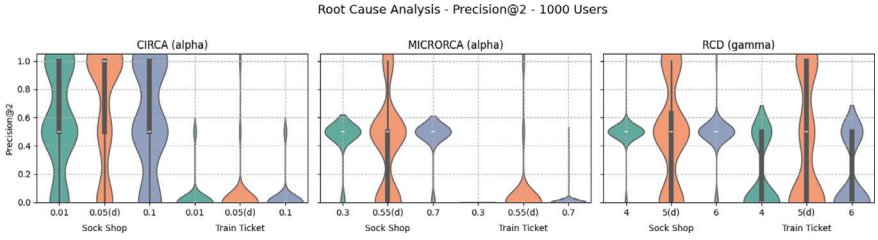


Fig. 31. PR@2 distribution for RCA algorithms across hyperparameter levels with a user load of 1000.

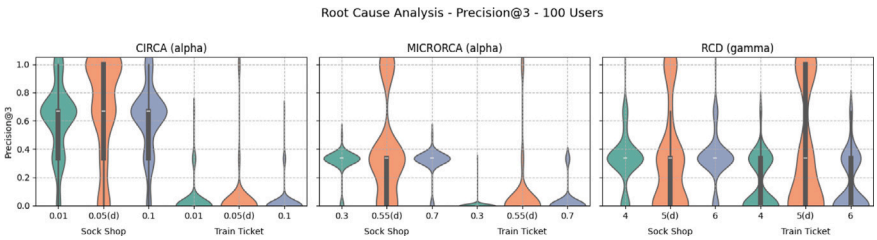


Fig. 32. PR@3 distribution for RCA algorithms across hyperparameter levels with a user load of 100.

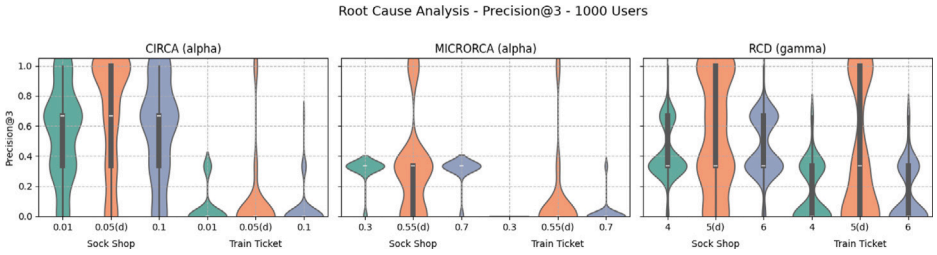


Fig. 33. PR@3 distribution for RCA algorithms across hyperparameter levels with a user load of 1000.

anomalies correctly or completely. In general, iForest is the best choice for detecting energy consumption anomalies, since it consistently had some of the best scores across the different AD metrics, combination of treatments, and datasets. KNN, LOF, and SVM performed better in terms of recall but were less precise, largely due to their sensitivity to hyperparameters that can cause the overfitting of anomalies. Future work could focus on incorporating hyperparameter tuning with the algorithms to improve their robustness.

RCA algorithms showed worse performance than AD algorithms, especially when considering the Train Ticket system. It is interesting to note that MicroRCA was originally developed for diagnosing performance issues and was applied to the Sock Shop system in its original article (Wu et al., 2020); there, it achieved a PR@1 of 0.89, outperforming the results observed in this study by far. We conjecture that there is a common issue for the relatively poor precision of all studied RCA algorithms: none of the considered RCA algorithms is specialized to energy consumption metrics. Energy anomalies, which can be caused by hardware, software, and other resource utilization, can be intermittent and less predictable than performance issues, making it harder for RCA algorithms to trace anomalies back to their root causes. Future work should aim to study the behavior of energy consumption anomalies and modify or create RCA algorithms that integrate energy-specific metrics into their processes. The results of the RCA algorithms for the Train Ticket system reveal another problem in the considered RCA algorithms. The precision of RCA algorithms in the Train Ticket system remained extremely low, even when considering the AP@2 and AP@3 metrics. This indicates that RCA algorithms struggle when tracing energy anomalies back to the correct service in larger and more complex systems due to the intricate interdependencies of their services. RCD is the only RCA algorithm that achieved moderate precision in both the Sock Shop and the Train Ticket systems, which may be due to its hierarchical learning and conditional independence tests that allow it to identify causal relationships more effectively.

Overall, our experiment indicates that stressing the selected services does not lead to significant propagation of energy anomalies to interconnected or downstream services. This limited propagation likely results from the inherent independence and resource isolation characteristic of microservice architectures. Such isolation simplifies root cause analysis, as performance anomalies and energy issues tend to remain localized within individual services rather than affecting the entire system. Nevertheless, further research is needed to explore different scenarios and architectural configurations that may exhibit propagation patterns. These insights would help refine RCA techniques specialized in integrating both performance and energy metrics at system level. These results emphasize the importance of choosing AD and RCA algorithms that are well-suited to energy-related metrics, as well as the complexity and scale of the target system, as the performance varies significantly based on the dataset and workload characteristics.

Regarding the services of Sock Shop, there are some cases where stressing the front-end service led to better performance of the AD algorithms than when the orders service is stressed. This pattern also occurs when considering RCA algorithms, especially MicroRCA. The most likely cause of this phenomenon is that while the front-end service is crucial for the user interactions with the system, the orders service works more in the background, where resource usage changes might be less obvious. It would be interesting to see in future work an investigation on whether choosing services to stress with different levels of importance for the system impacts the effectiveness of AD and RCA algorithms, especially in more complex systems like Train Ticket.

DevOps engineers should consider using algorithms like iForest for systems with high resource usage, where energy anomalies are more pronounced. However, they must balance between precision and recall depending on their system's tolerance for false positives versus undetected anomalies. In complex systems, lightweight RCA algorithms such as RCD can offer scalable solutions for diagnosing root causes of anomalies.

9. Threats to validity

9.1. Internal validity

Internal validity concerns the degree of certainty that the results from the experiment were not influenced by external factors or confounding factors. To mitigate the threat of a carryover effect, where consecutive runs can affect each other, there is a warm-up and cooldown periods to allow residual container resource consumption from previous runs to reach idle levels. The potential for order-related bias was also considered in this study, so the selection of the use case scenario, the stressed service, and the user load treatments is randomized for each run. AD algorithms such as BIRCH, iForest, SVM, LOF, and KNN were executed using default configurations from the PyCaret library, except for BIRCH, which was adjusted to label only 5% of the data as anomalies. Similarly, RCA algorithms like CIRCA, RCD, MicroRCA, and CausalRCA were configured based on their recommended parameters from the literature. However, not adjusting

the configurations to better analyze the collected data could have affected the overall AD and RCA results. Scaphandre was used to measure energy consumption and although widely used, it could cause data inconsistencies. Therefore, each experimental run was executed 30 times. These experiment design choices were made to minimize the influence of any potential external factors on the results. At the time of writing, there is no stressor specifically tailored to energy. This is why we decided to choose `stress-ng`, which is oriented towards (hardware-related) performance issues. Designing, developing, and validating an stressor fully tailored towards energy consumption is a promising research avenue for future researchers in the area of Green software engineering.

9.2. External validity

External validity has to do with the generalizability of the study's results to other systems and contexts. The study uses two microservice-based systems, Sock Shop and Train Ticket, which represent different levels of complexity. While these systems are widely used in research, they may not represent the behavior of microservice systems in the industry, such as systems with dynamic workloads. Future work could synthetically create microservice-based systems to investigate how these (Detti et al., 2023) affect the performance of AD and RCA algorithms. Moreover, although the usage scenarios were designed to reflect typical behaviors in the selected systems, they are quite similar, which may limit the generalizability of the findings. The usage scenarios for Sock Shop and Train Ticket both had a scenario where a user simply browses available items (e.g. socks, train tickets), and another which was a continuation of the previous scenario, involving completing the transaction (e.g. placing an order, booking a ticket). However, in real-world microservice systems, workloads can vary dynamically, which might result in different resource usage patterns and anomaly characteristics. To improve the generalizability of the findings, future studies could incorporate more diverse user scenarios and workload variations.

9.3. Construct validity

Construct validity evaluates whether experiment setup accurately measures the metrics of the experiment. The three-sigma rule was used to define anomalies in the ground truth. While this is a standard approach, it assumes a Gaussian distribution of metrics, which may not always hold true in real-world scenarios. Future studies that want to improve this would have to generate the ground truth of the AD algorithms and execute them again. In this experiment we are always stressing CPU, memory, and disk *at the same time*. Moreover, we collect power measurements from a RAPL-based tool (*i.e.,* Scaphandre), which assesses, with its own limitations, the total CPU consumption, consumption per CPU socket, and the consumption of the DRAM controller. This means that in the context of our experiment the inherent cause of an energy issue consists in an anomalous usage of hardware resources – that is commonly accepted in the field of Green software engineering (Guldner et al., 2024; Castor, 2024) – and that results refer to the energy consumption domains as estimated by RAPL. We are also aware that the energy consumption of a microservice-based system might be influenced by network-related activities; we do not consider this aspect in this study and we leave it for future work. In addition, it should be noted that if we consider formula (3), energy consumption does not depend only on the power consumed by hardware components, but it depends also on the execution time of the task at hand. In practical settings, power and time are not completely independent from each other; for example, a higher usage of the CPU might lead to longer queues for executing processes at the OS level, which in turn can lead to overall longer execution times, or saturating the available memory might lead to memory paging (or swapping) or even to the termination of certain processes, which again might result in longer execution times. There may also be cases where the higher energy consumption is not caused by the higher power usage, but is simply due to longer execution times (*e.g.,* a longer queue in the REST endpoint managing incoming requests of a service). In principle, in our experiment, we could have explored also this route, for example by systematically injecting idle waits in the microservices or throttling their REST endpoints; however, we discarded this option since it is not realistic to have time-related penalties without a specific (hardware, networking, or technically justifiable) cause. So, we decided to opt for a hardware-related cause for the energy issues for the sake of having a more realistic experiment.

9.4. Conclusion validity

Conclusion validity assesses the reliability of the conclusions from the study. Each treatment combination was executed 30 times, which is a reasonable sample size for experimental studies. This gives greater confidence in the results. Moreover, the precision, recall, and F-score metrics used to evaluate AD algorithms are well-established, as well as the AP@k scores for RCA algorithms. However, it is possible that there are other metrics that would give a better perspective of the results.

10. Conclusion

Using two target systems, Sock Shop and Train Ticket, the study investigates the effectiveness of various AD and RCA algorithms in identifying and diagnosing performance-related energy issues. The experiments evaluated the correctness, completeness, and accuracy of the algorithms under different stress scenarios and user loads. For anomaly detection, the iForest algorithm is consistently good at identifying energy consumption issues. For root cause analysis, our results show that some algorithms (RCA and CIRCA) tend to be able to trace the energy consumption issue back to the root cause with acceptable results, but their performance can be affected by the topology and complexity of the system.

Our datasets of normal and anomalous behavior for the Sock Shop and Train Ticket systems, including performance and energy consumption metrics, can be useful for future studies. For example, these datasets can be used as benchmarks to develop and compare new AD and RCA algorithms, ensuring reproducibility and comparability across studies. Further research is needed designing new RCA algorithms that (i) are scalable to larger datasets, (ii) have high precision, and (iii) support energy metrics as first-class entities. Moreover, our experiments can be extended by considering (i) additional microservice-based systems (*e.g.,* those presented in Gan et al. (2019a)), (ii) different topologies, and (iii) real-world workloads to validate the findings across a broader spectrum of use cases. Another interesting approach for future work would be combining some of the AD algorithms with RCA algorithms considered in this study — for example, having MicroRCA use iForest instead of the BIRCH algorithm.

CRediT authorship contribution statement

Berta Rodriguez Sanchez: Writing – original draft, Visualization, Validation, Software, Investigation, Data curation. **Luca Giamattei:** Writing – review & editing, Writing – original draft, Investigation. **Antonio Guerriero:** Writing – review & editing, Investigation, Conceptualization. **Roberto Pietrantuono:** Writing – review & editing, Supervision, Investigation, Funding acquisition, Conceptualization. **Ivano Malavolta:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This project has received funding from the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 871342 "uDEVOPS".

Data availability

Data will be made available on request.

References

Abdelrahman, Osama, Keikhosrokiani, Pantea, 2020. Assembly line anomaly detection and root cause analysis using machine learning. IEEE Access 8, 189661–189672.

Araújo, Gabriel, Barbosa, Vandirleya, Lima, Luiz Nelson, Sabino, Arthur, Brito, Carlos, Fé, Iure, Rego, Paulo, Choi, Eunmi, Min, Dugki, Nguyen, Tuan Anh, et al., 2024. Energy consumption in microservices architectures: a systematic literature review. IEEE Access.

Breunig, Markus M, Kriegel, Hans-Peter, Ng, Raymond T, Sander, Jörg, 2000. LOF: identifying density-based local outliers. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. pp. 93–104.

Caldiera, Victor R. Basili1 Gianluigi, Rombach, H. Dieter, 1994. The goal question metric approach. Encycl. Softw. Eng. 528–532.

Castor, Fernando, 2024. Estimating the energy footprint of software systems: a primer. arXiv preprint arXiv:2407.11611.

Chandola, Varun, Banerjee, Arindam, Kumar, Vipin, 2009. Anomaly detection: A survey. ACM Comput. Surv. 41 (3), 1–58.

Chen, Pengfei, Qi, Yong, Zheng, Pengfei, Hou, Di, 2014. Causeinfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems. In: IEEE INFOCOM 2014-IEEE Conference on Computer Communications. IEEE, pp. 1887–1895.

Cover, Thomas, Hart, Peter, 1967. Nearest neighbor pattern classification. IEEE Trans. Inform. Theory 13 (1), 21–27.

de Vries, Alex, 2023. The growing energy footprint of artificial intelligence. Joule 7 (10), 2191–2194.

Detti, Andrea, Funari, Ludovico, Petrucci, Luca, 2023. μBench: An open-source factory of benchmark microservice applications. IEEE Trans. Parallel Distrib. Syst. 34 (3), 968–980.

Di Francesco, Paolo, Lago, Patricia, Malavolta, Ivano, 2019. Architecting with microservices: A systematic mapping study. J. Syst. Softw. 150, 77–97.

Dinga, Madalina, Malavolta, Ivano, Giamattei, Luca, Guerriero, Antonio, Pietrantuono, Roberto, 2023. An empirical evaluation of the energy and performance overhead of monitoring tools on Docker-based systems. In: International Conference on Service-Oriented Computing. Springer, pp. 181–196.

Dragoni, Nicola, Giallorenzo, Saverio, Lafuente, Alberto Lluch, Mazzara, Manuel, Montesi, Fabrizio, Mustafin, Ruslan, Safina, Larisa, 2017. Microservices: yesterday, today, and tomorrow. Present. Ulterior Softw. Eng. 195–216.

Floroiu, Maximilian Stefan, Russo, Stefano, Giamattei, Luca, Guerriero, Antonio, Malavolta, Ivano, Pietrantuono, Roberto, 2024. Anomaly detection and root cause analysis of microservices energy consumption. In: 2024 IEEE International Conference on Web Services. ICWS, IEEE, pp. 590–600.

Freitag, Charlotte, Berners-Lee, Mike, Widdicks, Kelly, Knowles, Bran, Blair, Gordon S, Friday, Adrian, 2021. The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations. Patterns 2 (9).

Gan, Yu, Zhang, Yanqi, Cheng, Dailun, Shetty, Ankitha, Rathi, Priyal, Katarki, Nayan, Bruno, Ariana, Hu, Justin, Ritchken, Brian, Jackson, Brendon, et al., 2019a. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 3–18.

Gan, Yu, Zhang, Yanqi, Hu, Kelvin, Cheng, Dailun, He, Yuan, Pancholi, Meghna, Delimitrou, Christina, 2019b. Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 19–33.

Guldner, Achim, Bender, Rabea, Calero, Coral, Fernando, Giovanni S., Funke, Markus, Groger, Jens, Hilty, Lorenz M., Hornschemeyer, Julian, Hoffmann, Geerd-Dietger, Junger, Dennis, Kennes, Tom, Kreten, Sandro, Lago, Patricia, Mai, Franziska, Malavolta, Ivano, Murach, Julien, Obergoker, Kira, Schmidt, Benno, Tarara, Arne, De Veaugh-Geiss, Joseph, Weber, Sebastian, Westing, Max, Wohlgemuth, Volker, Naumann, Stefan, 2024. Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components-green software measurement model (GSMM). Future Gener. Comput. Syst. 155, 402–418.

Hameed, Abdul, Khoshkbarforoushha, Alireza, Ranjan, Rajiv, Jayaraman, Prem Prakash, Kolodziej, Joanna, Balaji, Pavan, Zeadally, Sherali, Malluhi, Qutaibah Marwan, Tziritas, Nikos, Vishnu, Abhinav, et al., 2016. A survey and taxonomy on energy efficient resource allocation techniques for cloud computing systems. Computing 98, 751–774.

Hochenbaum, Jordan, Vallis, Owen S., Kejariwal, Arun, 2017. Automatic anomaly detection in the cloud via statistical learning. arXiv preprint arXiv:1704.07706.

Ikram, Azam, Chakraborty, Sarthak, Mitra, Subrata, Saini, Shiv, Bagchi, Saurabh, Kocaoglu, Murat, 2022. Root cause analysis of failures in microservices through causal discovery. Adv. Neural Inf. Process. Syst. 35, 31158–31170.

Jaramillo, David, Nguyen, Duy V., Smart, Robert, 2016. Leveraging microservices architecture by using Docker technology. In: SoutheastCon 2016. IEEE, pp. 1–5.

Kalia, Anup K, Xiao, Jin, Krishna, Rahul, Sinha, Saurabh, Vukovic, Maja, Banerjee, Debasish, 2021. Mono2micro: a practical and effective tool for decomposing monolithic java applications to microservices. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1214–1224.

Kamat, Pooja, Sugandhi, Rekha, 2020. Anomaly detection for predictive maintenance in industry 4.0-A survey. In: E3S Web of Conferences. Vol. 170, EDP Sciences, p. 02007.

Khan, Kashif Nizam, Hirki, Mikael, Niemi, Tapio, Nurminen, Jukka K, Ou, Zhonghong, 2018. Rapl in action: Experiences in using rapl for power measurements. ACM Trans. Model. Perform. Eval. Comput. Syst. (TOMPECS) 3 (2), 1–26.

Klaver, Luuk, van der Knaap, Thijs, van der Geest, Johan, Harmsma, Edwin, van der Waaij, Bram, Pileggi, Paolo, 2021. Towards independent run-time cloud monitoring. In: Companion of the ACM/SPEC International Conference on Performance Engineering. pp. 21–26.

Latino, Mark A., Latino, Robert J., Latino, Kenneth C., 2019. Root Cause Analysis: Improving Performance for Bottom-Line Results. CRC Press.

Li, Mingjie, Li, Zeyan, Yin, Kanglin, Nie, Xiaohui, Zhang, Wenchi, Sui, Kaixin, Pei, Dan, 2022. Causal inference-based root cause analysis for online service systems with intervention recognition. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 3230–3240.

Lin, JinJin, Chen, Pengfei, Zheng, Zibin, 2018. Microscope: Pinpoint performance issues with causal graphs in micro-service environments. In: Service-Oriented Computing: 16th International Conference, ICSOC 2018, Hangzhou, China, November 12-15, 2018, Proceedings. Vol. 16, Springer, pp. 3–20.

Liu, Fei Tony, Ting, Kai Ming, Zhou, Zhi-Hua, 2008. Isolation forest. In: 2008 Eighth IEEE International Conference on Data Mining. IEEE, pp. 413–422.

Ma, Meng, Xu, Jingmin, Wang, Yuan, Chen, Pengfei, Zhang, Zonghua, Wang, Ping, 2020. Automap: Diagnose your microservice-based web applications automatically. In: Proceedings of the Web Conference 2020. pp. 246–258.

Nadareishvili, Irakli, Mitra, Ronnie, McLarty, Matt, Amundsen, Mike, 2016. Microservice Architecture: Aligning Principles, Practices, and Culture. " O'Reilly Media, Inc".

Newman, Sam, 2021. Building Microservices. " O'Reilly Media, Inc".

Peerally, Mohammad Farhad, Carr, Susan, Waring, Justin, Dixon-Woods, Mary, 2017. The problem with root cause analysis. BMJ Qual. Saf. 26 (5), 417–422.

Pinto, Gustavo, Castor, Fernando, 2017. Energy efficiency: a new concern for application software developers. Commun. ACM 60 (12), 68–75.

Rufino, Joao, Alam, Muhammad, Ferreira, Joaquim, Rehman, Abdur, Tsang, Kim Fung, 2017. Orchestration of containerized microservices for IIoT using Docker. In: 2017 IEEE International Conference on Industrial Technology. ICIT, IEEE, pp. 1532–1536.

Santos, Eddie Antonio, McLean, Carson, Solinas, Christopher, Hindle, Abram, 2018. How does Docker affect energy consumption? Evaluating workloads in and out of Docker containers. J. Syst. Softw. 146, 14–25.

Shahin, Mohammad, Chen, F Frank, Hosseinzadeh, Ali, Zand, Neda, 2023. Using machine learning and deep learning algorithms for downtime minimization in manufacturing systems: An early failure detection diagnostic service. Int. J. Adv. Manuf. Technol. 128 (9–10), 3857–3883.

Shaw, Mary, Garlan, David, 1996. Software Architecture: Perspectives on an Emerging Discipline. Prentice-Hall, Inc.

Shin, Hyun Joon, Eom, Dong-Hwan, Kim, Sung-Shick, 2005. One-class support vector machines—an application in machine fault detection and classification. Comput. Ind. Eng. 48 (2), 395–408.

Soldani, Jacopo, Brogi, Antonio, 2022. Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. ACM Comput. Surv. 55 (3), 1–39.

Soldani, Jacopo, Tamburri, Damian Andrew, Van Den Heuvel, Willem-Jan, 2018. The pains and gains of microservices: A systematic grey literature review. J. Syst. Softw. 146, 215–232.

Solé, Marc, Muntés-Mulero, Victor, Rana, Annie Ibrahim, Estrada, Giovani, 2017. Survey on models and techniques for root-cause analysis. arXiv preprint arXiv:1701.08546.

Steenwinckel, Bram, De Paepe, Dieter, Hautte, Sander Vanden, Heyvaert, Pieter, Bentefrit, Mohamed, Moens, Pieter, Dimou, Anastasia, Van Den Bossche, Bruno, De Turck, Filip, Van Hoecke, Sofie, et al., 2021. FLAGS: A methodology for adaptive anomaly detection and root cause analysis on sensor data streams by fusing expert knowledge with machine learning. Future Gener. Comput. Syst. 116, 30–48.

Torabi, Hasan, Mirtaheri, Seyedeh Leili, Greco, Sergio, 2023. Practical autoencoder based anomaly detection by using vector reconstruction error. Cybersecurity 6 (1), 1.

Vegas, Sira, Apa, Cecilia, Juristo, Natalia, 2015. Crossover designs in software engineering experiments: Benefits and perils. IEEE Trans. Softw. Eng. 42 (2), 120–135.

Vos, Sophie, Lago, Patricia, Verdecchia, Roberto, Heitlager, Ilja, 2022. Architectural tactics to optimize software for energy efficiency in the public cloud. In: 2022 International Conference on ICT for Sustainability. ICT4S, IEEE, pp. 77–87.

Wan, Xili, Guan, Xinjie, Wang, Tianjing, Bai, Guangwei, Choi, Baek-Yong, 2018. Application deployment using microservice and Docker containers: Framework and optimization. J. Netw. Comput. Appl. 119, 97–109.

Wang, Dongjie, Chen, Zhengzhang, Fu, Yanjie, Liu, Yanchi, Chen, Haifeng, 2023. Incremental causal graph learning for online root cause analysis. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 2269–2278.

Wu, Li, Tordsson, Johan, Elmroth, Erik, Kao, Odej, 2020. Microrca: Root cause localization of performance issues in microservices. In: NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.

Wu, Li, Tordsson, Johan, Elmroth, Erik, Kao, Odej, 2021. Causal inference techniques for microservice performance diagnosis: Evaluation and guiding recommendations. In: 2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems. ACSOS, IEEE, pp. 21–30.

Xin, Ruyue, Chen, Peng, Zhao, Zhiming, 2023. Causalrca: Causal inference based precise fine-grained root cause localization for microservice applications. J. Syst. Softw. 203, 111724.

Yu, Guangba, Chen, Pengfei, Li, Yufeng, Chen, Hongyang, Li, Xiaoyun, Zheng, Zibin, 2023. Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 553–565.

Zeng, Zhengran, Zhang, Yuqun, Xu, Yong, Ma, Minghua, Qiao, Bo, Zou, Wentao, Chen, Qingjun, Zhang, Meng, Zhang, Xu, Zhang, Hongyu, et al., 2023. Traceark: Towards actionable performance anomaly alerting for online service systems. In: 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, IEEE, pp. 258–269.

Zhang, Tian, Ramakrishnan, Raghu, Livny, Miron, 1996. BIRCH: an efficient data clustering method for very large databases. ACM Sigmod Rec. 25 (2), 103–114.

Zhou, Xiang, Peng, Xin, Xie, Tao, Sun, Jun, Xu, Chenjie, Ji, Chao, Zhao, Wenyun, 2018. Benchmarking microservice systems for software engineering research. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings. pp. 323–324.